



INGENIERÍA DE SISTEMAS COMPUTACIONALES

Análisis de la Capacidad de Respuesta Sanitaria ante la Incidencia Histórica de Dengue en el Perú

Autores:

Goyzueta Lamas, Krisstel Andrea

Guerra Barraza, Osvaldo

Hinostroza Rojas, Uriel Wilfredo

Parra De La Cruz, Alexander Fernando

Levano Suarez, Axel Jesus

Vladislavich Silva, Mia Milagros

Curso:

Base Datos Avanzada y Big Data

Docente:

Yonni Yovani Lopez Vitor

Lima – Perú

2026-2

Tabla de Contenido

CAPÍTULO I RELEVAMIENTO DEL DOMINIO Y SELECCIÓN DE DATOS	4
1.1 Descripción del Dominio y Selección de Datasets	4
1.2 Diagnóstico de Calidad de Datos	4
1.3 Declaración de Supuestos Técnicos y de Negocio	6
1.4 Requerimientos Funcionales	7
CAPÍTULO II DISEÑO E IMPLEMENTACIÓN DE OBJETOS DE AUTOMATIZACIÓN	8
2.1 Procedimientos Almacenados y Control de Transacciones	9
2.2 Triggers de Integridad y Auditoría	12
2.3 Función Escalar Reutilizable	14
2.4 Registro de Auditoría	15
CAPÍTULO III IMPLEMENTACIÓN DE POLÍTICAS DE SEGURIDAD Y CUMPLIMIENTO NORMATIVO	16
3.1 Gestión de Accesos y Roles	16
3.2 Política de Respaldo y Recuperación	17
3.3 Optimización de Rendimiento y Análisis de Plan de Ejecución	18
3.4. Lista de Verificación de Cumplimiento	20
CAPÍTULO IV EXPLORACIÓN DEL PARADIGMA NOSQL Y DECISIÓN DE INTEGRACIÓN	21
4.1 Migración a Modelo Orientado a Documentos (MongoDB)	21
4.2 Estructura BSON y Operaciones CRUD	21
4.3 Análisis Comparativo Arquitectónico: SQL Server vs MongoDB	24
4.4 Conclusión del Análisis	25
CAPÍTULO V DISEÑO DEL DATA WAREHOUSE Y PROCESOS ETL	25
5.1 Diseño del Data Warehouse	25
5.2 Modelo lógico y físico	27
5.3 Proceso ETL	30
5.4 Ejecución y validación del proceso ETL	33
5.5 Trazabilidad y control de la carga	35
5.6 Justificación del enfoque Kimball	36
5.7 Resultado del Data Warehouse	37

CAPÍTULO VI CONSTRUCCIÓN DE DASHBOARD BI Y ANÁLISIS OLAP	38
6.1. Objetivo del dashboard	38
6.2. Conexión y modelo de datos en Power BI	39
6.3. Indicadores clave de desempeño (KPI)	40
6.4. Diseño de las visualizaciones	42
6.5. Filtros interactivos y operaciones OLAP	44
6.6. Gobernanza y metadatos de los KPI	46
6.7. Preguntas de negocio y utilidad del dashboard	47
6.8. Conclusiones del capítulo	48
CAPÍTULO VII ANÁLISIS BIG DATA CON APACHE SPARK Y REFLEXIÓN DE ESCALABILIDAD	48
7.1. Inicialización, inspección y preparación del entorno	48
7.2. Evaluación de rendimiento y comparativa tecnológica	51
7.3. Análisis descriptivo y hallazgos epidemiológicos	55
7.4. Inspección física de ejecución y escalabilidad	58
7.5. Conclusiones del capítulo	59
CAPÍTULO VIII INTEGRACIÓN, REFLEXIÓN ÉTICA Y CONCLUSIONES DEL PROYECTO	60
8.1. Integración analítica: tecnología y salud pública	60
8.2. Reflexión ética en el manejo de datos	60
8.3. Transparencia y reproducibilidad	60
8.4. Conclusiones generales	60

CAPÍTULO I

RELEVAMIENTO DEL DOMINIO Y SELECCIÓN DE DATOS

1.1 Descripción del Dominio y Selección de Datasets

El proyecto, titulado "Análisis de la Capacidad de Respuesta Sanitaria ante la Incidencia Histórica de Dengue en el Perú (2000-2024)", tiene como objetivo evaluar si las zonas con mayor volumen histórico de casos cuentan con la infraestructura hospitalaria (IPRESS) suficiente para soportar futuras epidemias. Se integraron dos conjuntos de datos abiertos del MINSA; la relación entre ambos es exclusivamente geográfica, a través del campo ubigeo, ya que el dataset de dengue no contiene ningún identificador de establecimiento.

Nombre del Dataset	Fuente Original	Volumen Original	Granularidad
datos_abiertos_vigilancia_dengue_2000_2024.csv	CDC PERU / MINSA	1,029,42 1 filas	1 fila = 1 caso notificado de dengue.
IPRESS.csv	SUSALUD / MINSA	20,819 filas	1 fila = 1 establecimiento de salud único.

1.2 Diagnóstico de Calidad de Datos

La exploración se realizó mediante scripts en Python sobre los archivos completos, con verificación de integridad (conteo de líneas del archivo contra filas cargadas) para descartar pérdida de registros durante la lectura.

Dataset	Tipo de Problema	Columna(s) Afectada(s)	Registros Afectados	Acción Ejecutada en ETL
IPRESS	Duplicados por código	codigo_unico	52 filas (26 códigos)	Se conserva 1 fila por código, priorizando la que tiene coordenadas cuando difieren. Total final: 20,793
IPRESS	Coordenadas inválidas	norte, este	3 registros con valor "0,0"	Convertidas a nulo (no corresponden a una ubicación real)
IPRESS	Marcadores de nulo en texto	Múltiples columnas	140,797 celdas	Valores como "NULL", "ACTUALIZAR", "NO TIENE" convertidos a nulo real

IPRESS	Fecha centinela	inicio_de_actividad	1,117 registros	Convertida a nulo; columna tipada como fecha
IPRESS	Texto duplicado por digitación	clasificacion	32 filas	Valores repetidos dentro del mismo campo (ej. "X,X") deduplicados
IPRESS	Categoría no informativa	categoria	5,815 registros (27.9%), de los cuales 162 con internamiento	No se usa categoría en solitario para medir capacidad; se combina con tipo (ver 1.3, supuesto S4)
IPRESS	Nulos estructurales	camas, cota, norte/este, unidad_ejecutora, red, nro_doc_categorizacion	Entre 57.1% y 97.0% según columna	Conservados como nulos, sin imputar, para no alterar el catálogo
Dengue	Duplicados exactos	Todas las columnas	152,091 registros (14.8%)	Conservados; no hay evidencia suficiente para asumir que representan pacientes distintos (ver 1.3, supuesto S2)
Dengue	Edad inválida	edad (tipo_edad = Años)	14 registros (ej. valores de hasta 7 dígitos)	Convertida a nulo
Dengue	Unidad de edad inconsistente	edad, tipo_edad	6,701 registros (meses y días)	Convertidos a años; tipo_edad actualizado a "A"
Dengue	Texto corrupto (carácter especial perdido)	provincia, distrito	15,561 y 30,222 registros	Reparado cruzando por ubigeo con el catálogo IPRESS
Dengue	Texto corrupto no reparable	localidad	336 registros (de 53,909 nulos totales, 5.2%)	Documentado como limitación; no se usa en el análisis
Dengue	Nulos	localcod	148,616 registros (14.4%)	Vacío estructural en años 2000-2006; no imputado, no se usa en el análisis (que llega a nivel distrito)
Dengue	Nulos	díresa	23 registros (0.0%)	Sin impacto relevante; no se usa para cruces (codificación distinta a la de IPRESS)

Dengue	Cobertura geográfica	ubigeo	2,140 casos (0.2%) en 16 distritos sin ningún IPRESS registrado	No se trata como dato inválido; se documenta como hallazgo del análisis (ver 1.3, supuesto S3)
--------	----------------------	--------	---	--

1.3 Declaración de Supuestos Técnicos y de Negocio

Para garantizar la integridad del modelo dimensional a nivel nacional, se establecieron los siguientes supuestos rectores:

- **IPRESS inactivas o cerradas: no se eliminan del catálogo:** El dengue no contiene ningún identificador de establecimiento, por lo que la relación entre ambos datasets es exclusivamente geográfica (vía ubigeo). Conservar todas las IPRESS mantiene completo el conteo de infraestructura por distrito, sin depender de la posible antigüedad de cada establecimiento.
- **Temporalidad de la infraestructura:** el catálogo IPRESS refleja la situación actual (no incluye historial ni fecha de cierre), mientras que los casos de dengue abarcan 2000-2024. El análisis compara la incidencia histórica contra la infraestructura disponible hoy; no se reconstruye la capacidad de cada año.
- **Filas idénticas en dengue:** al no existir un identificador único de paciente, no se eliminan los registros idénticos. La tasa de duplicados exactos crece con el tamaño del grupo (año-semana-ubigeo-localidad): de 0% en grupos únicos a 36.7% en grupos de más de 50 casos, un patrón que no corresponde a coincidencia por azar (1.4% observado frente a 0.6% esperado). Esto no permite afirmar con certeza si se trata de pacientes distintos o de doble notificación en semanas de alta carga; por ello se conservan todos los registros y se planea incorporar, en la etapa del Data Warehouse, una medida de trazabilidad (n_ocurrencias_en_celda) en lugar de asumir una interpretación sin evidencia suficiente.

- **Definición de capacidad de respuesta:** el 27.9% de las IPRESS no tiene categoría asignada ("Sin Categoría"), y de ellas 162 son establecimientos con internamiento (incluye hospitales de las Fuerzas Armadas y de la Policía). Por ello, el criterio de "capacidad" para el KPI de infraestructura no puede basarse solo en categoría; se definirá en la etapa de modelado (Paso 5) como: tipo = "ESTABLECIMIENTO DE SALUD CON INTERNAMIENTO" o categoría $\geq$ I-3.
- **Cobertura geográfica sin infraestructura:** se identificaron 2,140 casos de dengue (0.2%) en 16 distritos que no registran ningún IPRESS, entre ellos Alto Trujillo (La Libertad, 1,164 casos solo en 2024). No se trata como un dato inválido ni se descarta: es en sí mismo un hallazgo relevante para la pregunta de negocio del proyecto (zonas con incidencia y sin infraestructura registrada).

1.4 Requerimientos Funcionales

El sistema de datos diseñado debe satisfacer las siguientes capacidades operativas para el análisis a nivel nacional:

- **Automatización de Ingesta Masiva:** carga ininterrumpida de 1,029,421 casos de dengue y 20,793 establecimientos IPRESS hacia SQL Server mediante Procedimientos Almacenados (TRY/CATCH), asegurando un log transaccional del 100% y conciliación de conteos entre origen y destino.
- **Seguridad Normativa (Ley N.º 29733):** implementación de 3 roles segregados en SQL Server y triggers de auditoría para proteger la trazabilidad de la data epidemiológica.
- **Análisis de Capacidad de Respuesta (BI):** construcción de un dashboard operativo con KPIs que crucen el volumen histórico de casos contra la densidad de infraestructura activa (según el criterio S4), permitiendo evaluar la vulnerabilidad sanitaria por departamento, provincia y distrito.

CAPÍTULO II

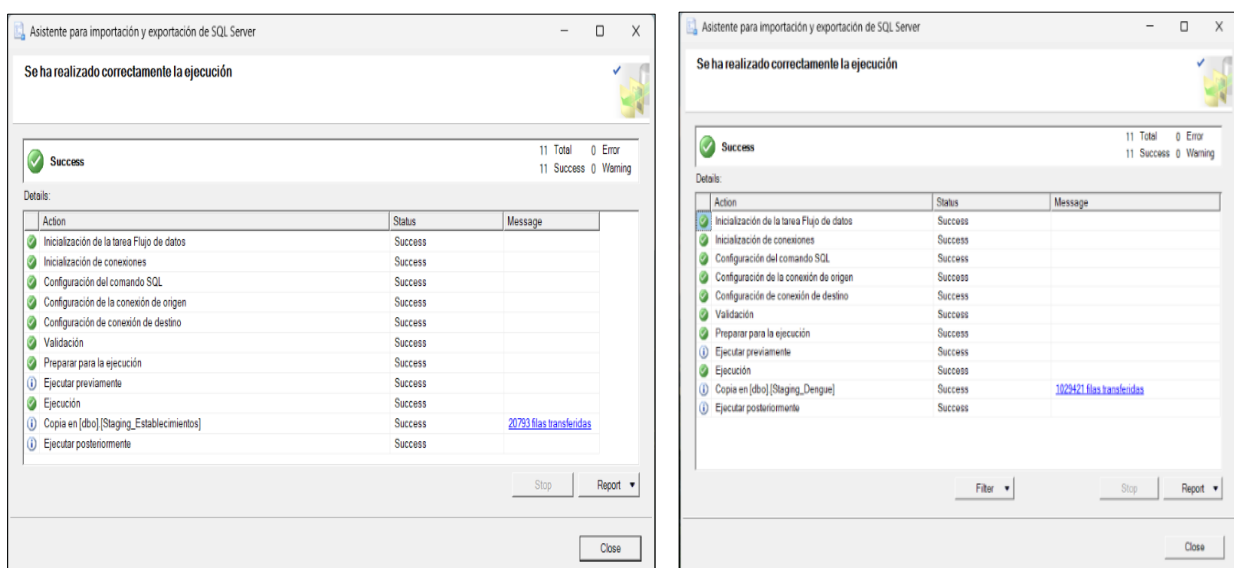
DISEÑO E IMPLEMENTACIÓN DE OBJETOS DE AUTOMATIZACIÓN

Una vez finalizada la limpieza y preparación de los archivos CSV, se procedió a implementar el modelo relacional en SQL Server. A partir de las estructuras y variables identificadas en los datasets, se realizó la separación de la información en entidades normalizadas, evitando mantener todos los atributos en una única tabla y estableciendo relaciones mediante claves primarias y foráneas.

En total, se crearon 9 entidades principales para representar la información del dominio: Institucion, Categoria, Clasificacion, Ubicacion, Establecimiento, Establecimiento_Clasificacion, Paciente, Caso_Dengue y Diagnostico. Adicionalmente, se implementaron 2 tablas de staging, Staging_Establecimientos y Staging_Dengue, utilizadas como área intermedia para recibir los datos provenientes de los archivos fuente antes de su transformación y distribución hacia las tablas normalizadas.

También se creó la tabla Historial_Auditoria, destinada a conservar los eventos generados por los mecanismos de auditoría y los errores registrados durante las operaciones de carga.

Con esta estructura implementada, se procedió al desarrollo de los objetos de automatización en SQL Server. Estos objetos permiten controlar la carga de información, mantener la integridad de los datos, registrar modificaciones y reutilizar reglas de negocio durante el procesamiento analítico.



2.1 Procedimientos Almacenados y Control de Transacciones

Para automatizar la ingesta de los datasets se crearon los procedimientos almacenados `sp_CargaEstablecimientos` y `sp_CargaCasosDengue`. El primero distribuye y transforma la información del catálogo de establecimientos de salud desde `Staging_Establecimientos` hacia las tablas normalizadas relacionadas con instituciones, categorías, clasificaciones, ubicaciones y establecimientos. El segundo procesa la información epidemiológica desde `Staging_Dengue` y genera los registros correspondientes en `Paciente`, `Caso_Dengue`, `Diagnostico` y `Ubicacion`.

Se eligió implementar esta lógica directamente en SQL Server en lugar de delegar la carga final a un script externo, debido a que los procedimientos permiten controlar las operaciones mediante transacciones administradas por el motor de base de datos.

Ambos procedimientos utilizan `BEGIN TRANSACTION`, `COMMIT TRANSACTION`, `ROLLBACK TRANSACTION` y bloques `TRY/CATCH`. De esta manera, ante una excepción durante la carga, se puede revertir la transacción y evitar que el proceso deje información parcialmente incorporada en las tablas finales. También se incorporó `SAVE TRANSACTION` como punto de guardado dentro del proceso de carga.

El bloque `CATCH` registra el error ocurrido en `Historial_Auditoria` y utiliza `THROW` para propagar la excepción al nivel de ejecución, permitiendo identificar la causa del fallo.

```
-- 1. Procedimiento para la carga e ingesta masiva de establecimientos de
salud (IPRESS) desde staging
CREATE OR ALTER PROCEDURE sp_CargaEstablecimientos
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        -- Catálogos: solo se insertan valores nuevos que aún no existan
        INSERT INTO Institucion (nombre)
        SELECT DISTINCT LTRIM(RTRIM(institucion))
        FROM Staging_Establecimientos s
        WHERE institucion IS NOT NULL
            AND NOT EXISTS (SELECT 1 FROM Institucion i WHERE i.nombre =
LTRIM(RTRIM(s.institucion)));

        INSERT INTO Categoria (nombre)
        SELECT DISTINCT LTRIM(RTRIM(categoria))
        FROM Staging_Establecimientos s
        WHERE categoria IS NOT NULL
            AND NOT EXISTS (SELECT 1 FROM Categoria c WHERE c.nombre =
LTRIM(RTRIM(s.categoria)));
    
```

```
-- 'clasificacion' puede traer varios valores separados por coma (ej.
"MEDICINA FISICA,REHABILITACION")
INSERT INTO Clasificacion (nombre)
SELECT DISTINCT LTRIM(RTRIM(v.value))
FROM Staging_Establecimientos s
CROSS APPLY STRING_SPLIT(s.clasificacion, ',') v
WHERE v.value IS NOT NULL AND LTRIM(RTRIM(v.value)) <> ''
AND NOT EXISTS (SELECT 1 FROM Clasificacion c WHERE c.nombre =
LTRIM(RTRIM(v.value)));

-- Ubicación: se agregan los ubigeos de IPRESS que aún no existan
INSERT INTO Ubicacion (ubigeo, departamento, provincia, distrito)
SELECT DISTINCT s.ubigeo, s.departamento, s.provincia, s.distrito
FROM Staging_Establecimientos s
WHERE s.ubigeo IS NOT NULL
AND NOT EXISTS (SELECT 1 FROM Ubicacion u WHERE u.ubigeo =
s.ubigeo);

SAVE TRANSACTION DespuesDeCatalogos;

-- Establecimiento: se hace el cast/validación fila por fila con
funciones seguras
INSERT INTO Establecimiento (
    codigo_unico, nombre, direccion, telefono, horario,
    fecha_inicio_actividad, director_medico, camas,
    longitud, latitud, cota, condicion,
    id_institucion, id_categoria, ubigeo
)
SELECT
    s.codigo_unico,
    s.nombre_del_establecimiento,
    s.direccion, s.telefono, s.horario,
    TRY_CONVERT(DATE, s.inicio_de_actividad, 103), -- 103 = formato
dd/mm/aaaa
    s.director_medico_y_o_responsable_de_la_atencion_de_salud,
    TRY_CAST(s.cameras AS INT),
    TRY_CAST(s.longitud AS DECIMAL(9,6)),
    TRY_CAST(s.latitud AS DECIMAL(9,6)),
    TRY_CAST(s.cota AS DECIMAL(10,3)),
    s.condicion,
    i.id_institucion,
    c.id_categoria,
    s.ubigeo
FROM Staging_Establecimientos s
JOIN Institucion i ON i.nombre = LTRIM(RTRIM(s.institucion))
JOIN Categoria c ON c.nombre = LTRIM(RTRIM(s.categoria))
WHERE NOT EXISTS (SELECT 1 FROM Establecimiento e WHERE e.codigo_unico
= s.codigo_unico);

-- Establecimiento_Clasificacion: se despliega la lista separada por
comas
INSERT INTO Establecimiento_Clasificacion (codigo_unico,
id_clasificacion)
SELECT DISTINCT s.codigo_unico, cl.id_clasificacion
FROM Staging_Establecimientos s
CROSS APPLY STRING_SPLIT(s.clasificacion, ',') v
JOIN Clasificacion cl ON cl.nombre = LTRIM(RTRIM(v.value))
WHERE NOT EXISTS (
    SELECT 1 FROM Establecimiento_Clasificacion ec
    WHERE ec.codigo_unico = s.codigo_unico AND ec.id_clasificacion =
cl.id_clasificacion
);

COMMIT TRANSACTION;
PRINT 'sp_CargaEstablecimientos: carga completada correctamente.';
```

```

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

    INSERT INTO Historial_Auditoria (accion_realizada, tabla_afectada,
detalle)
    VALUES ('INSERT', 'Establecimiento',
LEFT(CONCAT('Error en sp_CargaEstablecimientos: ',
ERROR_MESSAGE()), 450));

    THROW;
END CATCH
END;
GO

-- 2. Procedimiento para la carga e ingesta masiva de casos de Dengue desde
staging
CREATE OR ALTER PROCEDURE sp_CargaCasosDengue
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        -- Ubicación: se agregan los ubigeos que solo existen en Dengue (los
16 distritos sin ningún IPRESS registrado)
        INSERT INTO Ubicacion (ubigeo, departamento, provincia, distrito)
        SELECT DISTINCT s.ubigeo, s.departamento, s.provincia, s.distrito
        FROM Staging_Dengue s
        WHERE s.ubigeo IS NOT NULL
        AND NOT EXISTS (SELECT 1 FROM Ubicacion u WHERE u.ubigeo =
s.ubigeo);

        -- Diagnóstico: por si hubiera una combinación enfermedad/CIE-10 nueva
        INSERT INTO Diagnostico (tipo_diagnostico, codigo_cie10)
        SELECT DISTINCT s.enfermedad, s.diagnostic
        FROM Staging_Dengue s
        WHERE s.enfermedad IS NOT NULL
        AND NOT EXISTS (SELECT 1 FROM Diagnostico d WHERE d.tipo_diagnostico
= s.enfermedad);

        SAVE TRANSACTION DespuesDeCatalogosDengue;

        -- Paciente: una fila nueva por cada caso (sin buscar coincidencias),
usando MERGE + OUTPUT para
        -- capturar el id_paciente generado y poder enlazarlo con su fila de
origen en el siguiente paso.
        IF OBJECT_ID('tempdb..#MapaPaciente') IS NOT NULL DROP TABLE
#MapaPaciente;
        CREATE TABLE #MapaPaciente (staging_id INT, id_paciente INT);

        MERGE Paciente AS destino
        USING (
            SELECT staging_id,
TRY_CAST(edad AS DECIMAL(6,3)) AS edad,
ISNULL(tipo_edad, 'A') AS tipo_edad,
sexo
            FROM Staging_Dengue
        ) AS origen
        ON 1 = 0 --nunca hay coincidencia:
siempre inserta
        WHEN NOT MATCHED THEN
            INSERT (edad, tipo_edad, sexo)
            VALUES (origen.edad, origen.tipo_edad, origen.sexo)
    
```

```

OUTPUT origen.staging_id, inserted.id_paciente
INTO #MapaPaciente (staging_id, id_paciente);

-- Caso_Dengue: se arma usando el mapa recién generado
INSERT INTO Caso_Dengue (
    ano, semana, localidad, localcod, diresa,
    id_paciente, tipo_diagnostico, ubigeo
)
SELECT
    TRY_CAST(s.ano AS INT),
    TRY_CAST(s.semana AS INT),
    s.localidad, s.localcod, s.diresa,
    m.id_paciente,
    s.enfermedad,
    s.ubigeo
FROM Staging_Dengue s
JOIN #MapaPaciente m ON m.staging_id = s.staging_id;

COMMIT TRANSACTION;
PRINT 'sp_CargaCasosDengue: carga completada correctamente.';

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

    INSERT INTO Historial_Auditoria (accion_realizada, tabla_afectada,
detalle)
VALUES ('INSERT', 'Caso_Dengue',
LEFT(CONCAT('Error en sp_CargaCasosDengue: ',
ERROR_MESSAGE()), 450));

    THROW;
END CATCH
END;
GO

```

2.2 Triggers de Integridad y Auditoría

Para reforzar la trazabilidad y el control de modificaciones sobre la información de establecimientos, se implementaron dos triggers con responsabilidades diferenciadas.

El primero, `trg_Establecimiento_Auditoria`, funciona como mecanismo de auditoría DML sobre la tabla `Establecimiento`. Se ejecuta después de operaciones `INSERT`, `UPDATE` y `DELETE`, registrando en `Historial_Auditoria` la acción realizada, el código del establecimiento afectado y un detalle de la operación. En el caso de las actualizaciones, se conserva además la comparación de valores anteriores y nuevos para los atributos `camas` y `condicion`.

El segundo, `trg_Integridad_Establecimiento`, implementa una regla de integridad de negocio mediante `INSTEAD OF DELETE`. Antes de permitir la eliminación de un establecimiento, verifica su condición. Si la `IPRESS` se encuentra `EN FUNCIONAMIENTO`, el intento de eliminación es registrado en `Historial_Auditoria` y la operación es rechazada mediante `RAISERROR`. De esta manera se evita eliminar

directamente un establecimiento activo desde el catálogo sin pasar por la regla de integridad definida.

Se optó por implementar estos controles en la propia base de datos en lugar de depender exclusivamente de la aplicación que consuma los datos. De esta forma, las reglas permanecen asociadas al modelo y se aplican independientemente del mecanismo utilizado para acceder a las tablas.

```
--1. Trigger 1: Auditoria DML (sobre Establecimiento)
CREATE OR ALTER TRIGGER trg_Establecimiento_Auditoria
ON Establecimiento
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    -- INSERT: solo hay filas en 'inserted'
    IF EXISTS (SELECT 1 FROM inserted) AND NOT EXISTS (SELECT 1 FROM deleted)
    BEGIN
        INSERT INTO Historial_Auditoria (accion_realizada, tabla_afectada,
id_registro, detalle)
        SELECT 'INSERT', 'Establecimiento', codigo_unico,
            CONCAT('Nuevo establecimiento: ', nombre)
        FROM inserted;
    END

    -- DELETE: solo hay filas en 'deleted'
    IF EXISTS (SELECT 1 FROM deleted) AND NOT EXISTS (SELECT 1 FROM inserted)
    BEGIN
        INSERT INTO Historial_Auditoria (accion_realizada, tabla_afectada,
id_registro, detalle)
        SELECT 'DELETE', 'Establecimiento', codigo_unico,
            CONCAT('Establecimiento eliminado: ', nombre)
        FROM deleted;
    END

    -- UPDATE: hay filas tanto en 'inserted' como en 'deleted'
    IF EXISTS (SELECT 1 FROM inserted) AND EXISTS (SELECT 1 FROM deleted)
    BEGIN
        INSERT INTO Historial_Auditoria (accion_realizada, tabla_afectada,
id_registro, detalle)
        SELECT 'UPDATE', 'Establecimiento', i.codigo_unico,
            CONCAT('Camas: ', ISNULL(CONVERT(VARCHAR,d.camas), 'NULL'), ' -> ',
            ISNULL(CONVERT(VARCHAR,i.camas), 'NULL'),
            ' | Condición: ', ISNULL(d.condicion, 'NULL'), ' -> ',
            ISNULL(i.condicion, 'NULL'))
        FROM inserted i
        JOIN deleted d ON i.codigo_unico = d.codigo_unico;
    END
END;
GO

--2. Trigger 2: Integridad (sobre Establecimiento)
CREATE OR ALTER TRIGGER trg_Integridad_Establecimiento
ON Establecimiento
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;
```

```
-- Validar si están intentando borrar una IPRESS activa
IF EXISTS (SELECT 1 FROM deleted WHERE condicion = 'EN FUNCIONAMIENTO')
BEGIN
    -- Registramos la infracción en el log antes de abortar
    INSERT INTO Historial_Auditoria (accion_realizada, tabla_afectada,
id_registro, detalle)
    SELECT 'DELETE', 'Establecimiento', codigo_unico,
        'Bloqueado por Integridad: Intento de eliminar IPRESS en
funcionamiento'
    FROM deleted
    WHERE condicion = 'EN FUNCIONAMIENTO';

    RAISERROR('Violación de integridad: No se puede eliminar un
establecimiento que está EN FUNCIONAMIENTO.', 16, 1);
    RETURN;
END

-- Si el establecimiento no estaba activo (ej. INOPERATIVO), permitimos el
borrado
DELETE FROM Establecimiento
WHERE codigo_unico IN (SELECT codigo_unico FROM deleted);
END;
GO
```

2.3 Función Escalar Reutilizable

Para apoyar la fase analítica se creó la función escalar fn_GrupoEtario_MINSA, que recibe la edad del registro como un valor DECIMAL(6,3) y devuelve una clasificación etaria.

La función contempla cinco resultados posibles: Desconocido, Niños (0-11), Adolescentes (12-17), Adultos (18-59) y Adultos Mayores (60+).

La regla se implementó como una función reutilizable de SQL Server para que pueda aplicarse de forma consistente en diferentes consultas analíticas, evitando repetir la misma lógica mediante múltiples expresiones condicionales. Posteriormente, esta clasificación puede utilizarse para realizar agrupaciones y análisis epidemiológicos por grupo etario.

```
-- 3. Función Escalar para clasificar pacientes por grupo etario
CREATE OR ALTER FUNCTION fn_GrupoEtario_MINSA
(@edad DECIMAL(6,3)) RETURNS VARCHAR(25)
AS
BEGIN
    DECLARE @grupo VARCHAR(25);

    IF @edad IS NULL
        SET @grupo = 'Desconocido';
    ELSE IF @edad < 12
        SET @grupo = 'Niños (0-11)';
    ELSE IF @edad < 18
        SET @grupo = 'Adolescentes (12-17)';
    ELSE IF @edad < 60
        SET @grupo = 'Adultos (18-59)';
    ELSE
```

```
SET @grupo = 'Adultos Mayores (60+)';

RETURN @grupo;

END;
GO
```

2.4 Registro de Auditoría

Durante la implementación se realizaron pruebas funcionales sobre ambos triggers utilizando registros de la tabla Establecimiento. Se ejecutó un intento de eliminación sobre un establecimiento con condicion = 'EN FUNCIONAMIENTO', el cual fue correctamente bloqueado por trg_Integridad_Establecimiento, quedando registrado en Historial_Auditoria como una operación DELETE con el detalle "Bloqueado por Integridad: Intento de eliminar IPRESS en funcionamiento". Posteriormente, se ejecutaron dos operaciones UPDATE sobre los atributos camas y condicion de otro establecimiento, las cuales fueron capturadas por trg_Establecimiento_Auditoria, registrando en cada caso el valor anterior y el valor nuevo de ambos campos. Estas pruebas confirman que los dos triggers funcionan de manera independiente y consistente: el de integridad impide la eliminación de establecimientos activos, mientras que el de auditoría deja trazabilidad detallada de toda modificación realizada sobre la tabla.

12	
13	
14	SELECT * FROM Historial_Auditoria
15	
16	

133 %
10
0
↑
↓
Línea: 9, Carácter: 1

#	id_log	fecha_hora	usuario_db	accion_realizada	tabla_afectada	id_registro	detalle
1	1	2026-09-25 07:10:02.167	OGIUser	DELETE	Establecimiento	00000003	Bloqueado por Integridad: Intento de eliminar IPRESS en funcionamiento
2	2	2026-09-25 07:15:06.890	OGIUser	UPDATE	Establecimiento	00000001	Camas: 120 -> 50 Condición: INOPERATIVO -> EN FUNCIONAMIENTO
3	3	2026-09-25 07:15:06.890	OGIUser	UPDATE	Establecimiento	00000001	Camas: 50 -> 0 Condición: EN FUNCIONAMIENTO -> INOPERATIVO

CAPÍTULO III

IMPLEMENTACIÓN DE POLÍTICAS DE SEGURIDAD Y CUMPLIMIENTO NORMATIVO

3.1 Gestión de Accesos y Roles

Para dar cumplimiento a los lineamientos de la Ley N.º 29733 (Protección de Datos Personales) y los controles de acceso de la norma ISO/IEC 27001 (Control A.9), se implementó un modelo de seguridad basado en roles (RBAC). Esto garantiza que los usuarios del sistema accedan únicamente a la información estrictamente necesaria para sus funciones operativas o de fiscalización. Un hito crítico en este diseño normativo fue la protección de datos sensibles, aislando el acceso a la columna `director_medico`, para evitar la exposición de datos de carácter personal.

Se crearon tres roles diferenciados en la base de datos `DengueCapacidadSanitaria`:

- **Rol_Administrador:** Con control total (CONTROL) sobre la base de datos para tareas de mantenimiento, gestión de esquemas y administración general.
- **Rol_AnalistaDatos:** Con privilegios operativos de consulta (SELECT) sobre el esquema normalizado (Establecimiento, Caso_Dengue, catálogos y dimensión Ubicacion) y ejecución de funciones de negocio (como `fn_GrupoEtario_MINSA`). Se le aplicó una denegación explícita (DENY) a la columna `director_medico`.
- **Rol_Auditor:** Con privilegios exclusivos de solo lectura (SELECT) sobre la tabla de auditoría (`Historial_Auditoria`) y las tablas operativas, asegurando la trazabilidad independiente y protegiendo los datos contra modificaciones indebidas. También cuenta con restricción (DENY) sobre los datos personales.

```
-- 1. CREACION DE ROLES
CREATE ROLE Rol_Administrador;
CREATE ROLE Rol_AnalistaDatos;
CREATE ROLE Rol_Auditor;
GO

-- 2. ADMINISTRADOR (Control total sobre la base de datos)
GRANT CONTROL ON DATABASE::DengueCapacidadSanitaria TO Rol_Administrador;
GO

-- 3. ANALISTA DE DATOS (Consulta de información y ejecución de la función)
GRANT SELECT ON OBJECT::dbo.Establecimiento TO Rol_AnalistaDatos;
GRANT SELECT ON OBJECT::dbo.Caso_Dengue TO Rol_AnalistaDatos;
GRANT SELECT ON OBJECT::dbo.Paciente TO Rol_AnalistaDatos;
```

```
GRANT SELECT ON OBJECT::dbo.Ubicacion TO Rol_AnalistaDatos;
GRANT SELECT ON OBJECT::dbo.Institucion TO Rol_AnalistaDatos;
GRANT SELECT ON OBJECT::dbo.Categoria TO Rol_AnalistaDatos;
GRANT SELECT ON OBJECT::dbo.Clasificacion TO Rol_AnalistaDatos;
GRANT SELECT ON OBJECT::dbo.Diagnostico TO Rol_AnalistaDatos;
GRANT EXECUTE ON OBJECT::dbo.fn_GrupoEtario_MINSA TO Rol_AnalistaDatos;
--Restricción de acceso a datos personales (Ley N.º 29733)
DENY SELECT ON OBJECT::dbo.Establecimiento (director_medico) TO
Rol_AnalistaDatos;
GO

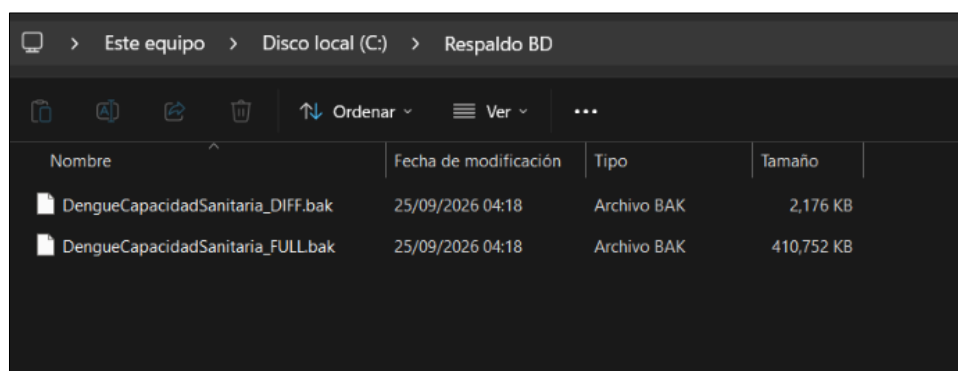
-- 4. AUDITOR (Consulta de datos y trazabilidad Sin permisos de modificación)
GRANT SELECT ON OBJECT::dbo.Historial_Auditoria TO Rol_Auditor;
GRANT SELECT ON OBJECT::dbo.Establecimiento TO Rol_Auditor;
GRANT SELECT ON OBJECT::dbo.Caso_Dengue TO Rol_Auditor;
--Restricción de acceso a datos personales (Ley N.º 29733)
DENY SELECT ON OBJECT::dbo.Establecimiento (director_medico) TO Rol_Auditor;
GO
```

3.2 Política de Respaldo y Recuperación

Alineado con el estándar ISO/IEC 27001 (Control A.12.3 - Copias de seguridad) para garantizar la disponibilidad y continuidad del servicio de salud, se diseñó una política de respaldo combinada basada en copias FULL (Completas) complementadas con respaldos DIFFERENTIAL (Diferenciales) almacenados en una ruta segura (C:\Respaldo BD\). Asimismo, se documentó y validó el procedimiento de restauración (RESTORE DATABASE), asegurando la recuperación efectiva de la base de datos ante posibles desastres.

```
-- 1. Respaldo Completo (FULL)
BACKUP DATABASE DengueCapacidadSanitaria
TO DISK = 'C:\Respaldo BD\DengueCapacidadSanitaria_FULL.bak'
WITH FORMAT, INIT,
NAME = 'DengueCapacidadSanitaria-Full Database Backup',
STATS = 10;
GO

-- 2. Respaldo Diferencial (DIFFERENTIAL)
BACKUP DATABASE DengueCapacidadSanitaria
TO DISK = 'C:\Respaldo BD\DengueCapacidadSanitaria_DIFF.bak'
WITH DIFFERENTIAL,
NAME = 'DengueCapacidadSanitaria-Differential Database Backup',
STATS = 10;
GO
```



```
USE master;
GO

-- Desconectar usuarios activos para evitar bloqueos durante la restauración
ALTER DATABASE DengueCapacidadSanitaria SET SINGLE_USER WITH ROLLBACK
IMMEDIATE;
GO

-- 1. Restaurar el Respaldo Completo en modo NORECOVERY (esperando el
diferencial)
RESTORE DATABASE DengueCapacidadSanitaria
FROM DISK = 'C:\Respaldo BD\DengueCapacidadSanitaria_FULL.bak'
WITH NORECOVERY, REPLACE, STATS = 10;
GO

-- 2. Restaurar el Respaldo Diferencial en modo RECOVERY (finalizando el
proceso)
RESTORE DATABASE DengueCapacidadSanitaria
FROM DISK = 'C:\Respaldo BD\DengueCapacidadSanitaria_DIFF.bak'
WITH RECOVERY, STATS = 10;
GO

-- Volver a habilitar el acceso a los usuarios
ALTER DATABASE DengueCapacidadSanitaria SET MULTI_USER;
GO
```

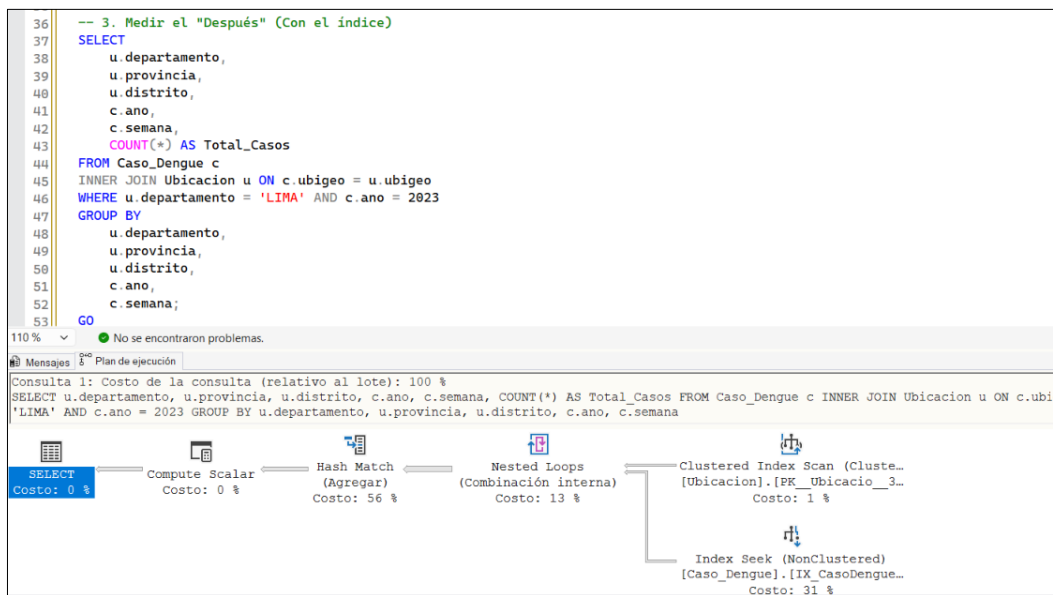
3.3 Optimización de Rendimiento y Análisis de Plan de Ejecución

Para evaluar el comportamiento de consultas críticas de análisis epidemiológico en el nuevo esquema normalizado (filtrado por departamento y año, lo que requiere un cruce de datos masivo con la dimensión geográfica), se midió el rendimiento antes y después de aplicar una estrategia de indexación orientada a evitar los recorridos completos de tabla (Table Scans).

A. Consulta Crítica Analítica:

```
-- Consulta crítica de ejemplo adaptada al esquema normalizado
SELECT
    u.departamento,
    u.provincia,
    u.districto,
    c.ano,
    c.semana,
    COUNT(*) AS Total_Casos
FROM Caso_Dengue c
INNER JOIN Ubicacion u ON c.ubigeo = u.ubigeo
WHERE u.departamento = 'LIMA' AND c.ano = 2023
GROUP BY
    u.departamento,
    u.provincia,
    u.districto,
    c.ano,
    c.semana;
GO
```


- **Después del Índice:** El plan de ejecución cambia a un operador optimizado de tipo Index Seek (NonClustered) acotado exclusivamente al nuevo índice creado (IX_CasoDengue_Ano_Ubigeo_Opt). Las lecturas lógicas sobre la tabla de hechos se reducen de manera drástica, optimizando la latencia de respuesta para los tableros de control de salud pública.



3.4. Lista de Verificación de Cumplimiento

En cumplimiento del Artículo 11 de la Ley N.º 29733, referente a la adopción de medidas de índole técnica, organizativa y legal para garantizar la seguridad de los datos personales, se presenta la siguiente matriz de verificación alineada con la norma ISO/IEC 27001:

Requisito / Medida Técnica (Art. 11 Ley 29733)	Estado	Mecanismo Implementado en la Base de Datos	Marco ISO 27001
Control de Acceso y Autenticación	Implementado	Creación de roles segregados (Rol_Administrador, Rol_AnalistaDatos, Rol_Auditor) basados en privilegios mínimos. Uso estricto de cláusulas DENY para proteger el campo director_medico.	A.9 Control de Acceso
Trazabilidad y Auditoría de Acciones	Implementado	Triggers de auditoría DML (trg_Establecimiento_Auditoria) y de integridad (trg_Integridad_Establecimiento) registrando estados anteriores/nuevos en Historial_Auditoria.	A.12.4 Registro y Monitoreo

Gestión de Respallos y Recuperación	Implementado	Política de respaldos automatizados (FULL + DIFFERENTIAL) y script de restauración validado desde ruta segura.	A.12.3 Copias de Seguridad
Integridad y Rendimiento de Datos	Implementado	Diseño relacional normalizado e índices optimizados compuestos (IX_CasoDengue_Ano_Ubigeo_Opt) sobre consultas analíticas críticas.	A.14 Seguridad en el Desarrollo

CAPÍTULO IV

EXPLORACIÓN DEL PARADIGMA NOSQL Y DECISIÓN DE INTEGRACIÓN

4.1 Migración a Modelo Orientado a Documentos (MongoDB)

Como parte de la evaluación arquitectónica para el manejo de los catálogos sanitarios, se implementó una base de datos NoSQL utilizando MongoDB. A diferencia del esquema altamente normalizado implementado en SQL Server, el modelo NoSQL se diseñó utilizando desnormalización por anidamiento.

Esta estrategia permitió agrupar toda la información de un establecimiento de salud (IPRESS) dentro de un único documento BSON (Binary JSON). Atributos que en el modelo relacional requerían uniones complejas (JOIN) con otras tablas, como la ubicación geográfica y la clasificación institucional, fueron integrados como subdocumentos dentro de la misma jerarquía.

4.2 Estructura BSON y Operaciones CRUD

Se configuró la base de datos DengueCapacidadSanitaria y la colección ipress. El diseño del documento elimina la latencia de lectura cruzada al empaquetar los datos relevantes en una sola estructura:

```
{
  "codigo_unico": "00000001",
  "nombre_del_establecimiento": "HOSPITAL NACIONAL DOS DE MAYO",
  "condicion": "EN FUNCIONAMIENTO",
  "camas": 120,
```

```
"ubicacion": {
  "departamento": "LIMA",
  "provincia": "LIMA",
  "distrito": "LIMA"
},
"clasificacion": {
  "institucion": "MINSA",
  "categoria": "III-1"
}
}
```

Sobre esta colección se ejecutaron y validaron operaciones DML (CRUD) utilizando mongosh, demostrando la capacidad del motor para filtrar subdocumentos mediante la notación de punto (ej. ubicacion.departamento).

```
> _MONGOSH
> use DengueCapacidadSanitaria
< switched to db DengueCapacidadSanitaria
> db.ipress.insertMany([
  {
    "codigo_unico": "00000001",
    "nombre_del_establecimiento": "HOSPITAL NACIONAL DOS DE MAYO",
    "condicion": "EN FUNCIONAMIENTO",
    "camas": 120,
    "ubicacion": {
      "departamento": "LIMA",
      "provincia": "LIMA",
      "distrito": "LIMA"
    },
    "clasificacion": {
      "institucion": "MINSA",
      "categoria": "III-1"
    }
  },
  {
    "codigo_unico": "00000002",
    "nombre_del_establecimiento": "POSTA MEDICA MIRONES",
    "condicion": "EN FUNCIONAMIENTO",
    "camas": 0,
    "ubicacion": {
      "departamento": "LIMA",
      "provincia": "LIMA",
      "distrito": "CERCADO DE LIMA"
    },
    "clasificacion": {
```

```
      "clasificacion": {
        "institucion": "ESSALUD",
        "categoria": "I-2"
      }
    },
    {
      "codigo_unico": "00000003",
      "nombre_del_establecimiento": "CLINICA SAN JUAN (PRUEBA)",
      "condicion": "INOPERATIVO",
      "camas": 10,
      "ubicacion": {
        "departamento": "CALLAO",
        "provincia": "CALLAO",
        "distrito": "CALLAO"
      },
      "clasificacion": {
        "institucion": "PRIVADO",
        "categoria": "II-1"
      }
    }
  ]
}< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ab68c81e0d8905a92ff57ca'),
    '1': ObjectId('6ab68c81e0d8905a92ff57cb'),
    '2': ObjectId('6ab68c81e0d8905a92ff57cc')
  }
}
```

```
> // Lee todos los establecimientos
db.ipress.find()

// Busca solo los establecimientos que están en LIMA (usando notación de punto)
db.ipress.find({ "ubicacion.departamento": "LIMA" })

// Busca establecimientos de ESSALUD sin camas (múltiples filtros)
db.ipress.find({
  "clasificacion.institucion": "ESSALUD",
  "camas": 0
})
< {
  _id: ObjectId('6ab68c81e0d8905a92ff57cb'),
  codigo_unico: '00000002',
  nombre_del_establecimiento: 'POSTA MEDICA MIRONES',
  condicion: 'EN FUNCIONAMIENTO',
  camas: 0,
  ubicacion: {
    departamento: 'LIMA',
    provincia: 'LIMA',
    distrito: 'CERCADO DE LIMA'
  },
  clasificacion: {
    institucion: 'ESSALUD',
    categoria: 'I-2'
  }
}
DengueCapacidadSanitaria>
```

```
> // Simula que la posta Mirones recibe 5 camas de internamiento temporal
db.ipress.updateOne(
  { "codigo_unico": "00000002" },
  { $set: { "camas": 5 } }
)

// Verificar la actualización
db.ipress.find({ "codigo_unico": "00000002" })
< {
  _id: ObjectId('6ab68c81e0d8905a92ff57cb'),
  codigo_unico: '00000002',
  nombre_del_establecimiento: 'POSTA MEDICA MIRONES',
  condicion: 'EN FUNCIONAMIENTO',
  camas: 5,
  ubicacion: {
    departamento: 'LIMA',
    provincia: 'LIMA',
    distrito: 'CERCADO DE LIMA'
  },
  clasificacion: {
    institucion: 'ESSALUD',
    categoria: 'I-2'
  }
}
DengueCapacidadSanitaria> |
```

```

> // Elimina la clínica de prueba que está inoperativa
db.ipress.deleteOne({ "codigo_unico": "00000003" })

// Verifica que se eliminó
db.ipress.find({ "codigo_unico": "00000003" })
<
DengueCapacidadSanitaria>
  
```

4.3 Análisis Comparativo Arquitectónico: SQL Server vs MongoDB

Con base en la experiencia del procesamiento de 1,029,421 casos de dengue y 20,793 registros de IPRESS, se establecen las siguientes diferencias estratégicas:

Criterio de Evaluación	Enfoque Relacional (SQL Server)	Enfoque NoSQL (MongoDB)
Estructura y Esquema	Rígido y normalizado: obliga a definir tipos de datos y relaciones (Foreign Keys) previamente. Ideal para evitar redundancia en los catálogos MINSA.	Flexible y desnormalizado: permite añadir nuevos campos (como equipos médicos) a un solo hospital sin alterar el esquema del resto de la colección.
Integridad Transaccional	Estricta (ACID): permite el uso de Triggers, bloqueos transaccionales (Rollbacks) e historial de auditoría exacto. Vital para el cumplimiento de la Ley N.º 29733.	Orientada al documento: las transacciones pueden realizarse a nivel de documento. Es eficiente, pero una auditoría detallada a nivel de campo puede requerir mayor esfuerzo de programación.
Rendimiento de Consultas	Optimizado para cruces: sobresaliente en consultas analíticas complejas, por ejemplo, agrupar casos de dengue por semana y cruzarlos con la capacidad hospitalaria regional, gracias a los Index Seek .	Optimizado para lecturas directas: adecuado cuando una aplicación web necesita cargar la ficha completa de un hospital, evitando operaciones JOIN .
Escalabilidad	Vertical: escalar el millón de registros de dengue requiere aumentar CPU, RAM o recursos del servidor central.	Horizontal: diseñado para fragmentar (Sharding) grandes volúmenes de datos en múltiples servidores de manera nativa.

4.4 Conclusión del Análisis

Para un entorno regido por el Ministerio de Salud, donde la trazabilidad legal de los datos, la auditoría médica y las agregaciones epidemiológicas complejas son críticas, SQL Server resulta ser la arquitectura principal idónea. Sin embargo, MongoDB se presenta como la tecnología complementaria perfecta para alimentar interfaces web de consulta rápida (como un directorio público de hospitales) debido a su flexibilidad documental y su entrega inmediata de datos anidados.

CAPÍTULO V

DISEÑO DEL DATA WAREHOUSE Y PROCESOS ETL

5.1 Diseño del Data Warehouse

Para el análisis de la capacidad de respuesta sanitaria frente a la incidencia histórica de dengue se implementó un Data Warehouse basado en el enfoque dimensional de Kimball. El modelo integra información epidemiológica proveniente de los casos de dengue con información de infraestructura sanitaria proveniente del catálogo de IPRESS del MINSA.

El objetivo del Data Warehouse es facilitar el análisis multidimensional de la incidencia histórica del dengue y su relación con la disponibilidad de infraestructura sanitaria a nivel geográfico.

La relación entre ambos dominios se establece mediante la ubicación geográfica representada por el código ubigeo. Esto se debe a que la fuente de vigilancia epidemiológica del dengue no contiene un identificador de establecimiento de salud asociado directamente a cada caso. Por esta razón, el modelo no establece una relación directa entre un caso de dengue y una IPRESS específica, sino una relación analítica a nivel de ubicación.

El diseño contiene cuatro dimensiones: Dim_Tiempo, Dim_Ubicacion, Dim_Diagnostico y Dim_Establecimiento

Y dos tablas de hechos: Fact_Casos_Dengue, Fact_Infraestructura

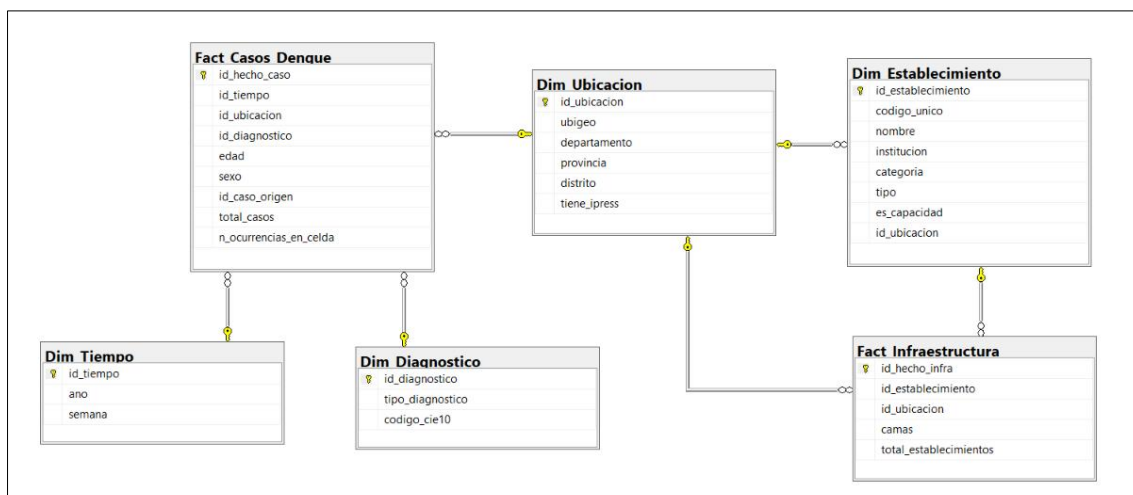
La separación de las dos tablas de hechos permite mantener diferenciados los dos procesos de negocio analizados: la ocurrencia de casos de dengue y la infraestructura sanitaria disponible.

Grano de las tablas de hechos: El grano de Fact_Casos_Dengue corresponde a un registro de dengue proveniente del modelo transaccional. Cada fila conserva la referencia al registro de origen mediante id_caso_origen y representa un caso con su ubicación, semana epidemiológica, diagnóstico, edad y sexo.

El grano de Fact_Infraestructura corresponde a un establecimiento de salud registrado en el catálogo de IPRESS. Cada registro conserva la cantidad de camas disponible cuando esta información se encuentra registrada y utiliza total_establecimientos = 1 para facilitar la agregación de establecimientos por ubicación.

De esta manera, el modelo permite realizar análisis como:

- cantidad de casos de dengue por año y semana epidemiológica;
- distribución de casos por departamento, provincia y distrito;
- distribución de casos según diagnóstico;
- cantidad de establecimientos de salud por ubicación;
- cantidad de camas registradas por ubicación;
- identificación de establecimientos considerados dentro del criterio de capacidad sanitaria;
- comparación entre incidencia histórica de dengue e infraestructura sanitaria disponible.



El diagrama muestra las dimensiones y tablas de hechos implementadas, así como las relaciones mediante claves sustitutas. Las tablas de hechos utilizan las dimensiones correspondientes para permitir consultas analíticas multidimensionales.

5.2 Modelo lógico y físico

El Data Warehouse se implementó en una base de datos independiente denominada DengueCapacidadSanitaria_DW, separada de la base de datos transaccional DengueCapacidadSanitaria.

La estructura física se diseñó mediante claves sustitutas (IDENTITY) para las dimensiones. Esto permite desacoplar las claves utilizadas en el modelo dimensional de las claves internas del modelo OLTP y facilita la construcción de consultas analíticas.

Dim_Tiempo: almacena el año y la semana epidemiológica. Se establece una restricción de unicidad sobre la combinación año y semana, evitando duplicidades en la dimensión. Esta dimensión permite analizar la evolución temporal de los casos de dengue por semana epidemiológica y año.

Dim_Ubicacion: almacena el ubigeo, departamento, provincia y distrito. Además, incorpora el atributo tiene_ipress, que indica si existe al menos un establecimiento de salud registrado para dicha ubicación. Este atributo se calcula durante el proceso ETL a partir de la existencia de establecimientos relacionados mediante ubigeo.

Dim_Diagnostico: almacena el tipo de diagnóstico y su correspondiente código CIE-10. Esta dimensión permite analizar los casos según el tipo de diagnóstico registrado en la fuente epidemiológica.

Dim_Establecimiento: almacena información descriptiva de cada IPRESS, incluyendo código único, nombre, institución, categoría, tipo y ubicación.

También incorpora el atributo es_capacidad, utilizado para representar el criterio de capacidad sanitaria definido en el proyecto. Un establecimiento es considerado dentro de este criterio cuando corresponde a un establecimiento de salud con internamiento o cuando su categoría corresponde a los niveles I-3, I-4, II-1, II-2, II-E, III-1, III-2 o III-E.

Fact_Casos_Dengue: contiene los registros epidemiológicos asociados a las dimensiones de tiempo, ubicación y diagnóstico.

Entre sus principales atributos se encuentran: id_tiempo, id_ubicacion, id_diagnostico, edad, sexo, id_caso_origen, total_casos, n_ocurrencias_en_celda

total_casos se establece en 1 para cada registro del hecho, permitiendo obtener cantidades mediante agregaciones.

El campo id_caso_origen mantiene la trazabilidad con el registro correspondiente del modelo transaccional.

El campo n_ocurrencias_en_celda registra la cantidad de ocurrencias existentes en la misma combinación de año, semana epidemiológica, ubicación y localidad. Su finalidad es principalmente analítica y de trazabilidad respecto al tratamiento de duplicados realizado durante la etapa de preparación de datos.

Fact_Infraestructura: representa la infraestructura sanitaria registrada en el catálogo de IPRESS.

Sus principales medidas son: camas; total_establecimientos.

El valor total_establecimientos = 1 permite contabilizar establecimientos mediante agregaciones agrupadas por ubicación u otras dimensiones relacionadas.

```
CREATE DATABASE DengueCapacidadSanitaria_DW;
GO

USE DengueCapacidadSanitaria_DW;

-- DIMENSIONES

CREATE TABLE Dim_Tiempo (
    id_tiempo INT IDENTITY(1,1) PRIMARY KEY,
    ano       SMALLINT NOT NULL,
    semana    TINYINT  NOT NULL,
    CONSTRAINT UQ_Dim_Tiempo UNIQUE (ano, semana)
);
GO

CREATE TABLE Dim_Ubicacion (
    id_ubicacion INT IDENTITY(1,1) PRIMARY KEY,
    ubigeo       CHAR(6) NOT NULL UNIQUE,
    departamento VARCHAR(60) NOT NULL,
    provincia    VARCHAR(60) NOT NULL,
    distrito     VARCHAR(60) NOT NULL,
    tiene_ipress BIT NOT NULL DEFAULT 0 -- se calcula en el ETL
    (pendiente resuelto)
);
```

```

GO

CREATE TABLE Dim_Diagnostico (
    id_diagnostico INT IDENTITY(1,1) PRIMARY KEY,
    tipo_diagnostico VARCHAR(50) NOT NULL UNIQUE,
    codigo_ciel0 VARCHAR(10) NOT NULL
);
GO

CREATE TABLE Dim_Establecimiento (
    id_establecimiento INT IDENTITY(1,1) PRIMARY KEY,
    codigo_unico CHAR(8) NOT NULL UNIQUE,
    nombre VARCHAR(150) NOT NULL,
    institucion VARCHAR(60) NULL,
    categoria VARCHAR(20) NULL,
    tipo VARCHAR(80) NULL,
    es_capacidad BIT NOT NULL DEFAULT 0, -- se calcula en
el ETL (pendiente resuelto)
    id_ubicacion INT NOT NULL,

    CONSTRAINT FK_DimEstablecimiento_Ubicacion FOREIGN KEY (id_ubicacion)
REFERENCES Dim_Ubicacion(id_ubicacion)
);
GO

-- HECHOS

CREATE TABLE Fact_Casos_Dengue (
    id_hecho_caso BIGINT IDENTITY(1,1) PRIMARY KEY,

    -- Llaves foráneas hacia las dimensiones
    id_tiempo INT NOT NULL,
    id_ubicacion INT NOT NULL,
    id_diagnostico INT NOT NULL,

    -- Atributos degenerados (no ameritan dimensión propia)
    edad DECIMAL(6,3) NULL,
    sexo CHAR(1) NULL,

    -- Trazabilidad hacia el sistema fuente (OLTP)
    id_caso_origen INT NOT NULL,

    -- Medidas
    total_casos INT NOT NULL DEFAULT 1, -- aditiva: SUM()
cuenta casos
    n_ocurrencias_en_celda INT NOT NULL, -- pendiente resuelto:
tamaño del grupo año-semana-ubigeo-localidad

    CONSTRAINT FK_FactCasos_Tiempo FOREIGN KEY (id_tiempo)
REFERENCES Dim_Tiempo(id_tiempo),
    CONSTRAINT FK_FactCasos_Ubicacion FOREIGN KEY (id_ubicacion)
REFERENCES Dim_Ubicacion(id_ubicacion),
    CONSTRAINT FK_FactCasos_Diagnostico FOREIGN KEY (id_diagnostico)
REFERENCES Dim_Diagnostico(id_diagnostico)
);
GO

CREATE TABLE Fact_Infraestructura (
    id_hecho_infra INT IDENTITY(1,1) PRIMARY KEY,

    id_establecimiento INT NOT NULL,
    id_ubicacion INT NOT NULL,

    camas INT NULL, -- NULL = sin dato, no
confundir con 0 camas

```

```
total_establecimientos INT NOT NULL DEFAULT 1,      -- aditiva: SUM()
cuenta establecimientos

CONSTRAINT FK_FactInfra_Establecimiento FOREIGN KEY (id_establecimiento)
REFERENCES Dim_Establecimiento(id_establecimiento),
CONSTRAINT FK_FactInfra_Ubicacion FOREIGN KEY (id_ubicacion)
REFERENCES Dim_Ubicacion(id_ubicacion)
);
GO
```

El script anterior corresponde a la implementación física de la base de datos DengueCapacidadSanitaria_DW, incluyendo las dimensiones, tablas de hechos, claves primarias y relaciones de integridad referencial.

5.3 Proceso ETL

El proceso ETL tiene como origen la base de datos transaccional DengueCapacidadSanitaria y como destino el Data Warehouse DengueCapacidadSanitaria_DW.

El proceso se implementó mediante el procedimiento almacenado sp_CargaDataWarehouse, que centraliza la carga de las dimensiones y tablas de hechos dentro de una única transacción.

La secuencia de carga implementada es la siguiente: Carga de Dim_Tiempo, Carga de Dim_Ubicacion, Carga de Dim_Diagnostico, Carga de Dim_Establecimiento, Carga de Fact_Infraestructura y finalmente Carga de Fact_Casos_Dengue.

Para evitar duplicaciones durante ejecuciones posteriores, las cargas utilizan condiciones NOT EXISTS basadas en las claves correspondientes.

El procedimiento utiliza una transacción explícita mediante BEGIN TRANSACTION, COMMIT TRANSACTION y ROLLBACK TRANSACTION. Además, implementa manejo de excepciones mediante TRY/CATCH. En caso de producirse un error, la transacción es revertida y el error se registra en Historial_Auditoria.

El procedimiento también registra la cantidad de filas incorporadas en cada dimensión y tabla de hechos. Esto permite mantener trazabilidad sobre la ejecución del proceso ETL.

Transformaciones realizadas: Durante el proceso ETL se aplican las siguientes transformaciones:

1. Generación de claves sustitutas para las dimensiones;
2. Identificación de ubicaciones con establecimientos registrados mediante tiene_ipress;
3. Transformación del catálogo de establecimientos en la dimensión dim_establecimiento;
4. Cálculo del atributo es_capacidad según el criterio definido para el proyecto;
5. Asociación de los casos de dengue con las dimensiones mediante ano, semana, ubigeo y diagnóstico;
6. Conservación del identificador de origen mediante id_caso_origen;
7. Cálculo de n_ocurrencias_en_celda mediante una función de ventana particionada por año, semana, ubicación y localidad;
8. Utilización de total_casos = 1 para representar cada registro epidemiológico en la tabla de hechos.

```
USE DengueCapacidadSanitaria_DW;
GO

CREATE OR ALTER PROCEDURE sp_CargaDataWarehouse
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @filas INT;

    BEGIN TRY
        BEGIN TRANSACTION;

        INSERT INTO Dim_Tiempo (ano, semana)
        SELECT DISTINCT c.ano, c.semana
        FROM DengueCapacidadSanitaria.dbo.Caso_Dengue c
        WHERE NOT EXISTS (SELECT 1 FROM Dim_Tiempo t WHERE t.ano = c.ano AND
t.semana = c.semana);
        SET @filas = @@ROWCOUNT;
        INSERT INTO DengueCapacidadSanitaria.dbo.Historial_Auditoria
(accion_realizada, tabla_afectada, detalle)
VALUES ('INSERT', 'Dim_Tiempo', CONCAT('ETL DW: ', @filas, ' filas
nuevas'));

        INSERT INTO Dim_Ubicacion (ubigeo, departamento, provincia, distrito,
tiene_ipress)
        SELECT u.ubigeo, u.departamento, u.provincia, u.distrito,
CASE WHEN EXISTS (SELECT 1 FROM
DengueCapacidadSanitaria.dbo.Establecimiento e WHERE e.ubigeo = u.ubigeo)
THEN 1 ELSE 0 END
```

```

FROM DengueCapacidadSanitaria.dbo.Ubicacion u
WHERE NOT EXISTS (SELECT 1 FROM Dim_Ubicacion d WHERE d.ubigeo =
u.ubigeo);
SET @filas = @@ROWCOUNT;
INSERT INTO DengueCapacidadSanitaria.dbo.Historial_Auditoria
(accion_realizada, tabla_afectada, detalle)
VALUES ('INSERT', 'Dim_Ubicacion', CONCAT('ETL DW: ', @filas, ' filas
nuevas'));

INSERT INTO Dim_Diagnostico (tipo_diagnostico, codigo_cie10)
SELECT d.tipo_diagnostico, d.codigo_cie10
FROM DengueCapacidadSanitaria.dbo.Diagnostico d
WHERE NOT EXISTS (SELECT 1 FROM Dim_Diagnostico x WHERE
x.tipo_diagnostico = d.tipo_diagnostico);
SET @filas = @@ROWCOUNT;
INSERT INTO DengueCapacidadSanitaria.dbo.Historial_Auditoria
(accion_realizada, tabla_afectada, detalle)
VALUES ('INSERT', 'Dim_Diagnostico', CONCAT('ETL DW: ', @filas, '
filas nuevas'));

INSERT INTO Dim_Establecimiento (codigo_unico, nombre, institucion,
categoria, tipo, es_capacidad, id_ubicacion)
SELECT
e.codigo_unico, e.nombre, i.nombre, c.nombre, e.tipo,
CASE WHEN e.tipo = 'ESTABLECIMIENTO DE SALUD CON INTERNAMIENTO'
OR c.nombre IN ('I-3', 'I-4', 'II-1', 'II-2', 'II-E', 'III-
1', 'III-2', 'III-E')
THEN 1 ELSE 0 END,
du.id_ubicacion
FROM DengueCapacidadSanitaria.dbo.Establecimiento e
JOIN DengueCapacidadSanitaria.dbo.Institucion i ON i.id_institucion =
e.id_institucion
JOIN DengueCapacidadSanitaria.dbo.Categoria c ON c.id_categoria =
e.id_categoria
JOIN Dim_Ubicacion du ON du.ubigeo = e.ubigeo
WHERE NOT EXISTS (SELECT 1 FROM Dim_Establecimiento x WHERE
x.codigo_unico = e.codigo_unico);
SET @filas = @@ROWCOUNT;
INSERT INTO DengueCapacidadSanitaria.dbo.Historial_Auditoria
(accion_realizada, tabla_afectada, detalle)
VALUES ('INSERT', 'Dim_Establecimiento', CONCAT('ETL DW: ', @filas, '
filas nuevas'));

INSERT INTO Fact_Infraestructura (id_establecimiento, id_ubicacion,
camas, total_establecimientos)
SELECT de.id_establecimiento, de.id_ubicacion, e.camas, 1
FROM DengueCapacidadSanitaria.dbo.Establecimiento e
JOIN Dim_Establecimiento de ON de.codigo_unico = e.codigo_unico
WHERE NOT EXISTS (SELECT 1 FROM Fact_Infraestructura f WHERE
f.id_establecimiento = de.id_establecimiento);
SET @filas = @@ROWCOUNT;
INSERT INTO DengueCapacidadSanitaria.dbo.Historial_Auditoria
(accion_realizada, tabla_afectada, detalle)
VALUES ('INSERT', 'Fact_Infraestructura', CONCAT('ETL DW: ', @filas, '
filas nuevas'));

INSERT INTO Fact_Casos_Dengue (
id_tiempo, id_ubicacion, id_diagnostico, edad, sexo,
id_caso_origen, total_casos, n_ocurrencias_en_celda
)
SELECT
dt.id_tiempo, du.id_ubicacion, dd.id_diagnostico,
p.edad, p.sexo, c.id_caso, 1,
COUNT(*) OVER (PARTITION BY c.ano, c.semana, c.ubigeo,
c.localidad)
FROM DengueCapacidadSanitaria.dbo.Caso_Dengue c

```

```

        JOIN DengueCapacidadSanitaria.dbo.Paciente p ON p.id_paciente =
        c.id_paciente
        JOIN Dim_Tiempo dt ON dt.ano = c.ano AND dt.semana = c.semana
        JOIN Dim_Ubicacion du ON du.ubigeo = c.ubigeo
        JOIN Dim_Diagnostico dd ON dd.tipo_diagnostico = c.tipo_diagnostico
        WHERE NOT EXISTS (SELECT 1 FROM Fact_Casos_Dengue f WHERE
        f.id_caso_origen = c.id_caso);
        SET @filas = @@ROWCOUNT;
        INSERT INTO DengueCapacidadSanitaria.dbo.Historial_Auditoria
        (accion_realizada, tabla_afectada, detalle)
        VALUES ('INSERT', 'Fact_Casos_Dengue', CONCAT('ETL DW: ', @filas, '
        filas nuevas'));

        COMMIT TRANSACTION;
        PRINT 'sp_CargaDataWarehouse: ETL completado correctamente.';

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

        INSERT INTO DengueCapacidadSanitaria.dbo.Historial_Auditoria
        (accion_realizada, tabla_afectada, detalle)
        VALUES ('INSERT', 'DataWarehouse',
        LEFT(CONCAT('Error en sp_CargaDataWarehouse: ',
        ERROR_MESSAGE()), 450));

        THROW;
    END CATCH
END;
GO

```

La implementación permite ejecutar la carga de las dimensiones y hechos de manera controlada, registrando en el modelo transaccional las cantidades de filas incorporadas durante cada etapa.

5.4 Ejecución y validación del proceso ETL

La ejecución del procedimiento sp_CargaDataWarehouse permitió cargar la información correspondiente al período analizado.

Los resultados registrados en Historial_Auditoria fueron los siguientes:

Objeto cargado	Filas nuevas
Dim_Tiempo	1,305
Dim_Ubicacion	1,877
Dim_Diagnostico	3
Dim_Establecimiento	20,793
Fact_Infraestructura	20,793
Fact_Casos_Dengue	1,029,421

Estos resultados evidencian que la carga incorporó las 1,029,421 filas de casos de dengue disponibles en el modelo transaccional y los 20,793 establecimientos previamente depurados durante el proceso de preparación de datos.

```

226 -- Verificar el log
227 SELECT TOP 10 * FROM DengueCapacidadSanitaria.dbo.Historial_Auditoria
228 WHERE tabla_afectada LIKE 'Dim_%' OR tabla_afectada LIKE 'Fact_%'
229 ORDER BY id_log DESC;

```

	id_log	fecha_hora	usuario_db	accion_realizada	tabla_afectada	id_registro	detalle
1	20819	2026-10-04 04:30:21.993	OG\User	INSERT	Fact_Casos_Dengue	NULL	ETL DW: 1029421 filas nuevas
2	20818	2026-10-04 04:30:06.423	OG\User	INSERT	Fact_Infraestructura	NULL	ETL DW: 20793 filas nuevas
3	20817	2026-10-04 04:30:06.077	OG\User	INSERT	Dim_Establecimiento	NULL	ETL DW: 20793 filas nuevas
4	20816	2026-10-04 04:30:05.223	OG\User	INSERT	Dim_Diagnostico	NULL	ETL DW: 3 filas nuevas
5	20815	2026-10-04 04:30:05.223	OG\User	INSERT	Dim_Ubicacion	NULL	ETL DW: 1877 filas nuevas
6	20814	2026-10-04 04:30:05.067	OG\User	INSERT	Dim_Tiempo	NULL	ETL DW: 1305 filas nuevas

La consulta permite verificar las cantidades de filas registradas por el procedimiento sp_CargaDataWarehouse para cada dimensión y tabla de hechos.

Validación de cantidades en el Data Warehouse: Posteriormente se verificó directamente el contenido de las dimensiones y tablas de hechos mediante consultas de conteo.

Los resultados obtenidos fueron:

```

159 ---Verificación del DW paso 5
160 USE DengueCapacidadSanitaria_DW;
161 GO
162
163
164 SELECT 'Dim_Tiempo' t, COUNT(*) n FROM Dim_Tiempo
165 UNION ALL SELECT 'Dim_Ubicacion', COUNT(*) FROM Dim_Ubicacion
166 UNION ALL SELECT 'Dim_Diagnostico', COUNT(*) FROM Dim_Diagnostico
167 UNION ALL SELECT 'Dim_Establecimiento', COUNT(*) FROM Dim_Establecimiento
168 UNION ALL SELECT 'Fact_Infraestructura', COUNT(*) FROM Fact_Infraestructura
169 UNION ALL SELECT 'Fact_Casos_Dengue', COUNT(*) FROM Fact_Casos_Dengue;

```

	t	n
1	Dim_Tiempo	1305
2	Dim_Ubicacion	1877
3	Dim_Diagnostico	3
4	Dim_Establecimiento	20793
5	Fact_Infraestructura	20793
6	Fact_Casos_Dengue	1029421

La cantidad de registros de Fact_Casos_Dengue coincide con la cantidad de casos cargados en el modelo transaccional, mientras que Fact_Infraestructura coincide con los 20,793 establecimientos depurados utilizados como fuente para el Data Warehouse.

La validación permite comprobar que las cargas principales del Data Warehouse fueron ejecutadas y que las cantidades obtenidas corresponden con los registros esperados desde el modelo transaccional.

5.5 Trazabilidad y control de la carga

El diseño mantiene mecanismos de trazabilidad entre el modelo transaccional y el modelo dimensional. En Fact_Casos_Dengue, el campo id_caso_origen conserva el identificador del caso existente en la tabla Caso_Dengue. Esto permite relacionar un registro del Data Warehouse con su registro de origen.

Asimismo, el procedimiento ETL registra en Historial_Auditoria la ejecución de cada etapa de carga y la cantidad de filas incorporadas.

Esta estrategia permite verificar tanto el volumen transferido como el origen de los registros epidemiológicos, manteniendo la trazabilidad requerida para el análisis.

La utilización de NOT EXISTS también evita que una segunda ejecución del procedimiento vuelva a insertar registros que ya fueron cargados, utilizando las claves de negocio correspondientes como mecanismo de control.

Limitación respecto a cargas incrementales: El cálculo de n_ocurrencias_en_celda se realiza mediante: COUNT(*) OVER (PARTITION BY ano, semana, ubigeo, localidad).

Durante la carga completa utilizada en este proyecto, el cálculo considera los 1,029,421 registros disponibles y permite determinar las ocurrencias correspondientes a cada celda.

Sin embargo, si posteriormente se modificara el proceso para cargar únicamente registros nuevos, el cálculo realizado sobre el nuevo lote no representaría automáticamente las ocurrencias acumuladas históricamente de la misma celda. Para un escenario incremental sería necesario recalcular o actualizar este valor para las celdas afectadas.

Esta limitación no afecta los resultados del presente proyecto, debido a que el Data Warehouse fue construido mediante una carga completa del período 2000–2024.

5.6 Justificación del enfoque Kimball

Se seleccionó el enfoque dimensional de Kimball debido a que el objetivo principal del proyecto es facilitar el análisis de información para responder preguntas de negocio relacionadas con la incidencia del dengue y la capacidad de infraestructura sanitaria.

El modelo permite organizar los datos alrededor de procesos de negocio claramente definidos. En este caso, se separaron dos procesos:

1. ocurrencia de casos de dengue;
2. disponibilidad de infraestructura sanitaria.

Esta separación se representa mediante Fact_Casos_Dengue y Fact_Infraestructura, mientras que las dimensiones permiten analizar los hechos desde diferentes perspectivas.

El enfoque también facilita la utilización posterior de herramientas de inteligencia de negocios, debido a que las tablas de hechos contienen medidas que pueden ser agregadas y las dimensiones proporcionan los diferentes niveles de análisis.

Entre las principales ventajas obtenidas se encuentran:

- facilidad para realizar consultas analíticas;
- separación clara de los procesos de negocio;
- utilización de dimensiones para análisis temporal, geográfico y epidemiológico;
- incorporación de medidas como cantidad de casos, camas y establecimientos;
- facilidad para construir indicadores para herramientas de BI;
- menor complejidad de consulta respecto de un modelo completamente normalizado.

Comparación con el enfoque Inmon: la elección de Kimball frente a Inmon se sustenta en dos criterios técnicos diferenciadores.

El primer criterio es la dirección de construcción del modelo. Kimball sigue un enfoque *bottom-up*: se parte directamente de un proceso de negocio concreto (en este caso, casos de dengue e infraestructura sanitaria) y se construye un modelo dimensional orientado a responder sus preguntas analíticas de forma inmediata. Inmon, en cambio, sigue un enfoque *top-down*: exige modelar primero un repositorio corporativo centralizado y normalizado (equivalente en complejidad al modelo OLTP ya implementado en DengueCapacidadSanitaria), del cual recién después se derivan los data marts departamentales. Para este proyecto, dado que ya existe un modelo transaccional normalizado que cumple ese rol de repositorio fuente, construir un segundo nivel de normalización corporativa antes del modelo analítico habría sido una duplicación de esfuerzo sin beneficio adicional para el alcance definido.

El segundo criterio es el alcance y tiempo de implementación. El enfoque de Inmon está pensado para entornos empresariales con múltiples áreas de negocio y necesidades de integración a largo plazo, lo que implica mayores tiempos de modelado, gobernanza de datos y mantenimiento. El alcance de este proyecto está acotado a dos procesos de negocio relacionados por una única dimensión geográfica compartida, por lo que el enfoque Kimball permite llegar a un modelo funcional, documentado y cargado en un tiempo consistente con el cronograma académico del curso, sin sacrificar la correcta identificación de hechos y dimensiones.

Por ambos criterios, Kimball resulta más adecuado para el alcance académico y analítico del proyecto, especialmente considerando que el modelo será utilizado posteriormente para construir indicadores y visualizaciones de inteligencia de negocios.

5.7 Resultado del Data Warehouse

Como resultado de la implementación se obtuvo un Data Warehouse dimensional compuesto por cuatro dimensiones y dos tablas de hechos.

La carga final contiene: 1,305 registros en Dim_Tiempo; 1,877 registros en Dim_Ubicacion; 3 registros en Dim_Diagnostico; 20,793 registros en Dim_Establecimiento; 20,793 registros en Fact_Infraestructura; 1,029,421 registros en Fact_Casos_Dengue.

El modelo permite integrar la información epidemiológica y sanitaria desde una perspectiva analítica común, utilizando ubigeo como elemento geográfico de integración.

De esta manera, el Data Warehouse constituye la base para la siguiente etapa del proyecto, en la cual los datos podrán ser utilizados para construir indicadores y visualizaciones orientadas a evaluar la relación entre la incidencia histórica del dengue y la infraestructura sanitaria disponible en las distintas zonas del Perú.

CAPÍTULO VI

CONSTRUCCIÓN DE DASHBOARD BI Y ANÁLISIS OLAP

6.1. Objetivo del dashboard

Se desarrolló un dashboard en Microsoft Power BI Desktop con el propósito de analizar la incidencia histórica del dengue en el Perú durante el periodo 2000–2024 y relacionarla con la infraestructura sanitaria registrada en el catálogo de Instituciones Prestadoras de Servicios de Salud (IPRESS).

El dashboard utiliza como fuente el Data Warehouse DengueCapacidadSanitaria_DW, implementado mediante un modelo dimensional basado en el enfoque de Kimball. Este modelo integra información epidemiológica, geográfica, temporal y de establecimientos de salud para facilitar el análisis de los datos desde distintas perspectivas.

El objetivo principal es proporcionar una herramienta visual que permita identificar los periodos y departamentos con mayor cantidad de casos registrados, explorar las diferencias geográficas y evaluar la relación entre la incidencia histórica del dengue y la infraestructura sanitaria disponible en el catálogo actual.

Es importante precisar que los datos de dengue abarcan los años 2000–2024, mientras que el catálogo de IPRESS representa la infraestructura registrada actualmente. Por ello, los resultados permiten realizar una comparación analítica entre ambas fuentes, pero no reconstruir la capacidad sanitaria que existía en cada año histórico.

6.2. Conexión y modelo de datos en Power BI

Para construir el dashboard se estableció una conexión desde Power BI Desktop hacia la base de datos DengueCapacidadSanitaria_DW, seleccionando las siguientes seis tablas:

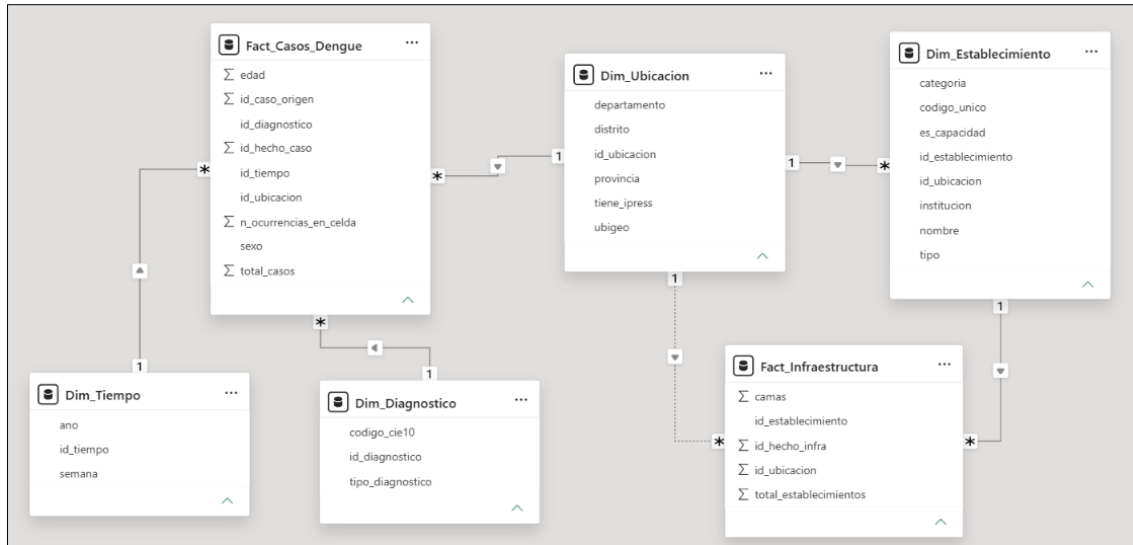
- Dim_Tiempo
- Dim_Ubicacion
- Dim_Diagnostico
- Dim_Establecimiento
- Fact_Casos_Dengue
- Fact_Infraestructura

Las tablas dimensionales permiten describir los hechos desde diferentes perspectivas. Dim_Tiempo proporciona el contexto temporal; Dim_Ubicacion permite el análisis territorial; Dim_Diagnostico clasifica los registros epidemiológicos; y Dim_Establecimiento contiene información sobre las características y la clasificación de las IPRESS.

Por su parte, Fact_Casos_Dengue contiene las medidas relacionadas con los casos registrados, mientras que Fact_Infraestructura contiene las medidas relacionadas con establecimientos, camas y capacidad sanitaria definida para el proyecto.

Las relaciones del modelo permiten conectar los hechos con sus dimensiones correspondientes. La relación entre la información epidemiológica y la infraestructura se realiza principalmente mediante la dimensión geográfica, utilizando la ubicación identificada por el código UBIGEO. No existe una relación directa entre un caso de dengue y una IPRESS específica, debido a que el dataset epidemiológico no contiene un identificador del establecimiento donde fue atendido cada paciente.

La relación entre Fact_Infraestructura y Dim_Ubicacion se mantiene inactiva para evitar rutas de filtrado ambiguas, dado que la infraestructura también puede relacionarse con la ubicación mediante Dim_Establecimiento.



6.3. Indicadores clave de desempeño (KPI)

Para facilitar la interpretación de los datos se definieron cinco medidas DAX. Estas medidas permiten cuantificar los casos de dengue, el número de establecimientos registrados, las camas disponibles en el catálogo y la cantidad de IPRESS que cumplen el criterio de capacidad sanitaria establecido para el proyecto.

Medidas DAX implementadas

a) Total Casos: Calcula la suma de los casos registrados en la tabla de hechos epidemiológicos.

Total Casos = SUM(Fact_Casos_Dengue[total_casos])

b) Total IPRESS: Calcula el número total de establecimientos representados en la tabla de hechos de infraestructura.

Total IPRESS = SUM(Fact_Infraestructura[total_establecimientos])

c) Camas Registradas: Calcula la suma de las camas registradas en el catálogo de infraestructura sanitaria. El resultado depende de los registros que contengan valores de camas válidos.

Camas Registradas = SUM(Fact_Infraestructura[camas])

d) IPRESS Capacidad: Calcula el número de establecimientos que cumplen el criterio de capacidad sanitaria definido para el proyecto. Se considera que una IPRESS cumple el criterio si corresponde a un establecimiento de salud con internamiento o si pertenece a una categoría igual o superior a I-3, de acuerdo con las categorías contempladas en el modelo.

$\text{IPRESS Capacidad} = \text{CALCULATE}([\text{Total IPRESS}], \text{Dim_Establecimiento}[\text{es_capacidad}] = \text{TRUE}())$

e) Casos por IPRESS: Calcula la relación entre el total de casos de dengue y el número de establecimientos considerados en el contexto de filtros vigente.

$\text{Casos por IPRESS} = \text{DIVIDE}([\text{Total Casos}], [\text{Total IPRESS}], 0)$
--

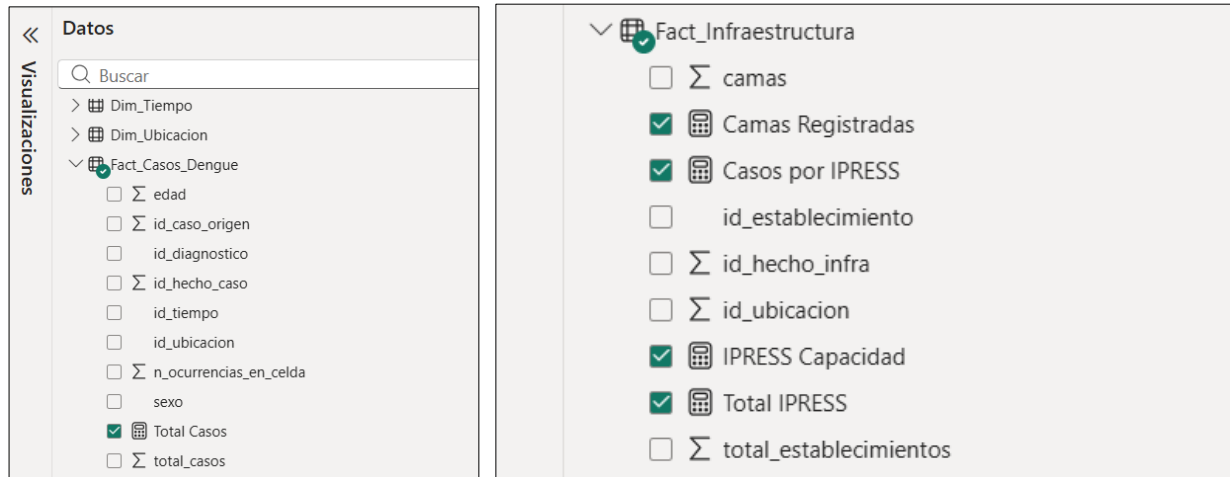
Esta última medida representa una razón analítica entre casos históricos y establecimientos registrados. No debe interpretarse como la cantidad real de pacientes atendidos por cada establecimiento ni como una prueba aislada de insuficiencia de capacidad sanitaria.

Resultados generales de los indicadores

Sin aplicar filtros que restrinjan los datos, las medidas proporcionan los siguientes resultados de referencia:

Indicador	Resultado general	Interpretación
Total Casos	1,029,421	Total de registros epidemiológicos considerados en el análisis
Total IPRESS	20,793	Establecimientos registrados después de la depuración
Camas Registradas	29,701	Total de camas registradas en el catálogo
IPRESS Capacidad	3,794	Establecimientos que cumplen el criterio de capacidad definido
Casos por IPRESS	49.51 aproximadamente	Razón entre casos históricos y establecimientos registrados

Los resultados corresponden al conjunto de datos procesado y validado durante el desarrollo del proyecto. El indicador IPRESS Capacidad se basa en una regla de clasificación definida para el análisis y no constituye una evaluación clínica u oficial de la capacidad real de cada establecimiento.



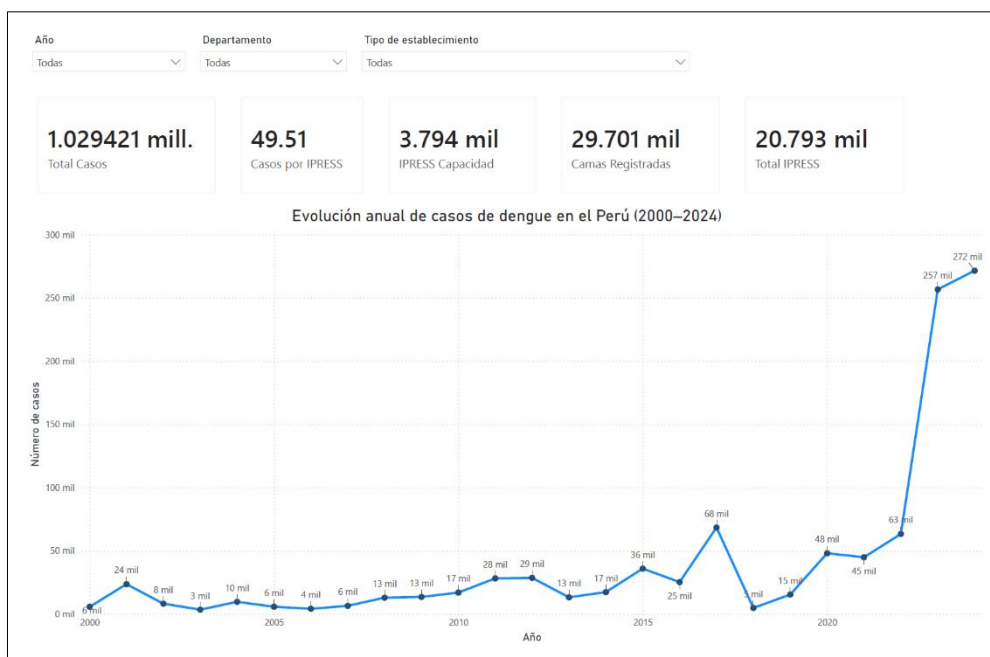
6.4. Diseño de las visualizaciones

El dashboard se organizó en tres páginas para presentar la información desde perspectivas complementarias y evitar la repetición innecesaria de visualizaciones.

Página 1 - Evolución anual de casos de dengue en el Perú (2000–2024): La primera página contiene un gráfico de líneas que representa la evolución anual de los casos de dengue registrados entre 2000 y 2024. Para construirlo se utilizó el campo Dim_Tiempo[ano] en el eje horizontal y la medida [Total Casos] en el eje vertical.

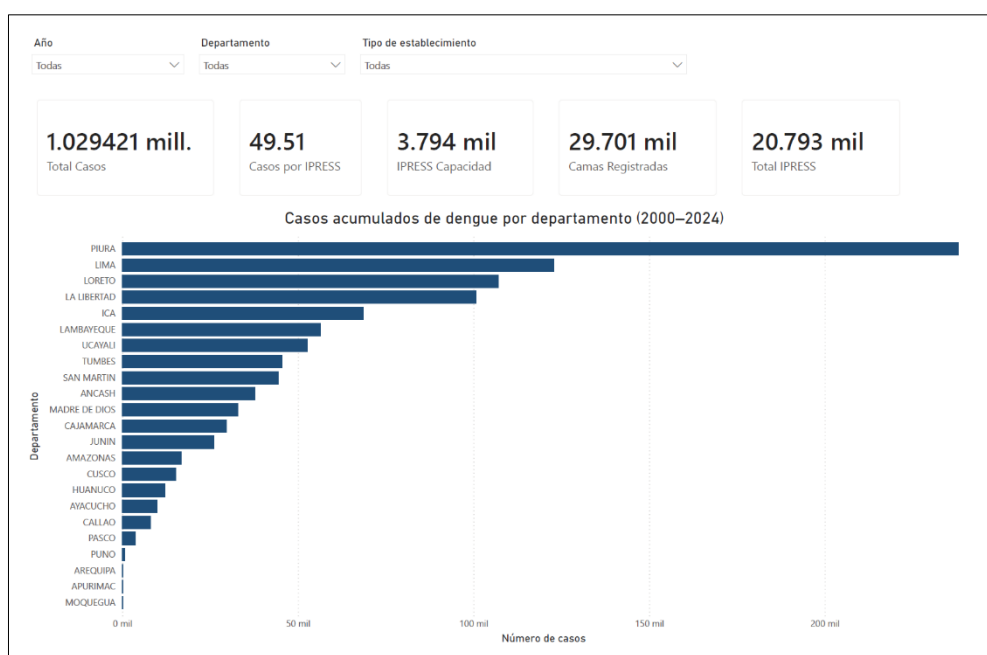
Esta visualización facilita la identificación de incrementos y disminuciones en los registros epidemiológicos a lo largo del periodo analizado. Permite reconocer los años de mayor incidencia registrada y explorar las variaciones temporales mediante las opciones interactivas del gráfico.

En esta página se incorporaron las tarjetas Total Casos y Casos por IPRESS, que resumen el volumen de casos y la relación entre los casos registrados y las IPRESS consideradas.



Página 2 - Casos acumulados de dengue por departamento (2000–2024): La segunda página presenta la distribución de los casos acumulados por departamento mediante un gráfico de barras horizontales. Se utiliza `Dim_Ubicacion[departamento]` como categoría geográfica y `[Total Casos]` como medida.

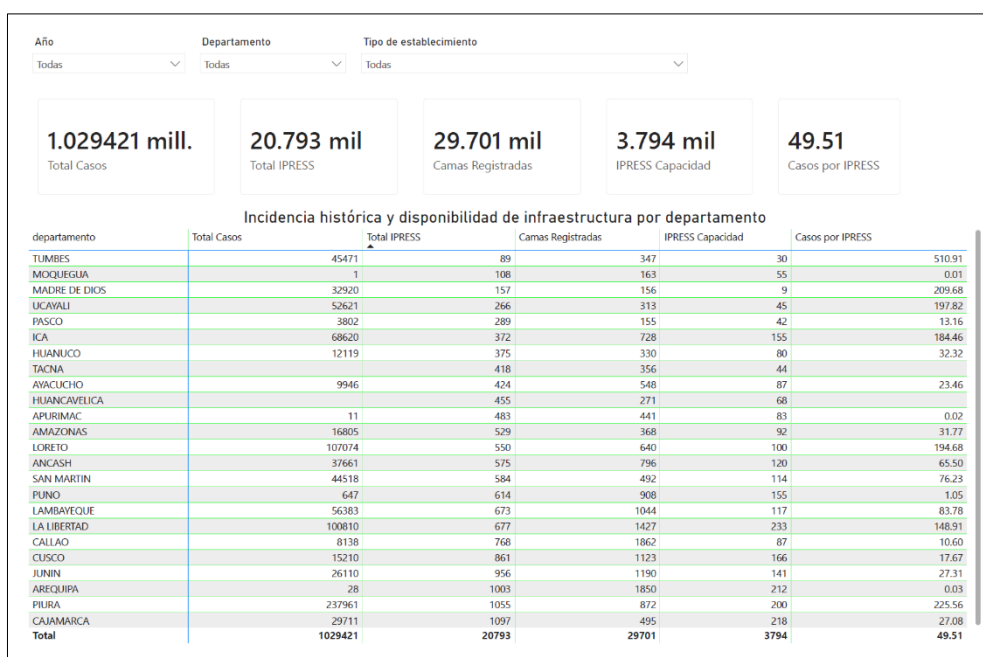
Esta visualización permite comparar el volumen acumulado de registros entre departamentos e identificar las zonas que concentran una mayor cantidad de casos durante el periodo estudiado.



Página 3 - Incidencia histórica y disponibilidad de infraestructura por departamento:

La tercera página complementa el análisis epidemiológico con información de infraestructura sanitaria. Su propósito es facilitar la comparación entre los casos históricos registrados y los establecimientos incluidos en el catálogo de IPRESS.

La interpretación conjunta permite identificar departamentos donde la relación entre casos y establecimientos registrados es elevada y examinar la distribución territorial de la infraestructura. También se pueden analizar las camas registradas y los establecimientos que cumplen el criterio de capacidad definido para el proyecto.



6.5. Filtros interactivos y operaciones OLAP

Para facilitar la exploración de los datos se incorporaron tres segmentaciones de datos en las páginas del dashboard. Estas permiten modificar el contexto de análisis sin tener que crear visualizaciones adicionales para cada posible combinación de criterios.

Los filtros implementados son los siguientes:

Filtro	Campo utilizado	Propósito
Año	Dim_Tiempo[ano]	Analizar los casos por año o por periodos seleccionados
Departamento	Dim_Ubicacion[departamento]	Restringir y comparar los resultados por ámbito geográfico

Tipo de establecimiento	Dim_Establecimiento[tipo]	Examinar la infraestructura según la clase de IPRESS
-------------------------	---------------------------	--

Los filtros se sincronizaron entre las páginas para conservar los criterios de selección durante la navegación. Esto permite realizar comparaciones con un contexto de análisis coherente y facilita la exploración interactiva de los resultados.

Se implementaron las siguientes operaciones de análisis multidimensional (OLAP):

Slice y dice: la selección de un año y un departamento permite restringir el conjunto de datos a un periodo y un ámbito geográfico determinados. La combinación de filtros permite explorar diferentes subconjuntos de información.

Drill-down geográfico: la estructura de ubicación permite profundizar desde departamento hacia provincia y distrito cuando se utiliza la jerarquía geográfica correspondiente en una visualización compatible.

Comparación temporal: la selección de años facilita examinar las variaciones en los registros de dengue entre diferentes periodos.



6.6. Gobernanza y metadatos de los KPI

Para facilitar la interpretación y el mantenimiento del dashboard, se documentaron los metadatos principales de cada indicador. Esta documentación permite identificar su fuente, fórmula, unidad de medida, finalidad y limitaciones de interpretación.

KPI	Fuente	Fórmula de cálculo	Unidad	Responsable
Total Casos	Fact_Casos_Dengue	Suma de total_casos	Casos registrados	Equipo del proyecto / responsable de análisis de datos
Total IPRESS	Fact_Infraestructura	Suma de total_establecimientos	Establecimientos	Equipo del proyecto / responsable de análisis de datos
Camas Registradas	Fact_Infraestructura	Suma de camas	Camas registradas	Equipo del proyecto / responsable de análisis de datos
IPRESS Capacidad	Fact_Infraestructura y Dim_Establecimiento	Suma de establecimientos que cumplen es_capacidad = TRUE	Establecimientos	Equipo del proyecto / responsable de análisis de datos
Casos por IPRESS	Medidas Total Casos y Total IPRESS	Total Casos / Total IPRESS	Razón de casos por establecimiento	Equipo del proyecto / responsable de análisis de datos

Fuente de los datos: Ministerio de Salud del Perú (MINSA), mediante los conjuntos de datos abiertos de vigilancia epidemiológica del dengue y del catálogo de IPRESS utilizados en el proyecto.

Reglas de interpretación y calidad:

1. Los casos corresponden a los registros epidemiológicos procesados para el periodo 2000–2024.
2. Los establecimientos corresponden al catálogo de IPRESS depurado e integrado en el Data Warehouse.
3. Las camas registradas no necesariamente representan camas operativas o disponibles en un momento específico.

4. La clasificación es _capacidad corresponde al criterio técnico adoptado en el proyecto; no reemplaza una evaluación oficial de la capacidad asistencial.
5. El indicador Casos por IPRESS es una razón analítica y no una medición directa de la carga de pacientes atendidos.
6. Los filtros y las relaciones del modelo determinan el contexto de cálculo de las medidas. Por tanto, cada resultado debe interpretarse según los filtros activos y el tipo de indicador analizado.

La actualización de los indicadores depende de la actualización de las fuentes, de la ejecución del proceso ETL y de la actualización de los datos en Power BI.

6.7. Preguntas de negocio y utilidad del dashboard

El dashboard fue diseñado para responder preguntas relevantes para el análisis epidemiológico y la planificación sanitaria:

- ¿Cómo ha variado el número de casos registrados de dengue entre 2000 y 2024?
- ¿Qué departamentos concentran el mayor número acumulado de casos durante el periodo?
- ¿Cómo cambia la distribución de los casos al seleccionar un año o un departamento?
- ¿Cuántas IPRESS se encuentran registradas en el catálogo y cuántas cumplen el criterio de capacidad definido?
- ¿Cómo se distribuyen las camas registradas y los establecimientos según el territorio y el tipo de IPRESS?
- ¿En qué departamentos se observa una relación elevada entre casos históricos e infraestructura registrada?

Las respuestas obtenidas permiten identificar patrones y orientar análisis posteriores. Sin embargo, los resultados deben utilizarse como evidencia descriptiva y exploratoria, no como prueba causal de que una determinada cantidad de establecimientos sea suficiente o insuficiente para atender la demanda sanitaria.

6.8. Conclusiones del capítulo

La implementación del dashboard en Power BI permitió integrar los resultados del Data Warehouse en un entorno visual e interactivo. Las cinco medidas DAX definidas facilitan el cálculo de indicadores epidemiológicos y de infraestructura, mientras que las tres páginas organizan el análisis temporal, geográfico y de capacidad sanitaria.

La incorporación de filtros de año, departamento y tipo de establecimiento permite explorar los datos mediante operaciones OLAP y examinar distintos subconjuntos de información. Además, la documentación de los metadatos y las limitaciones de los KPI contribuye a una interpretación más transparente de los resultados.

Finalmente, la solución demuestra la utilidad de integrar SQL Server, un modelo dimensional y Power BI para analizar conjuntos de datos sanitarios de gran volumen. La principal limitación interpretativa es que la incidencia epidemiológica es histórica, mientras que la infraestructura corresponde al catálogo actual. Por ello, las conclusiones deben centrarse en la relación descriptiva entre ambas fuentes y no en atribuir una capacidad de atención histórica que los datos disponibles no permiten medir directamente.

CAPÍTULO VII

ANÁLISIS BIG DATA CON APACHE SPARK Y REFLEXIÓN DE ESCALABILIDAD

7.1. Inicialización, inspección y preparación del entorno

Para iniciar el procesamiento, se levantó una sesión de Apache Spark (versión 4.2.0) en modo local (local[*]), asignando los 12 núcleos lógicos del equipo. En esta fase se ejecutó la ingesta y preparación de los conjuntos de datos:

- **Carga y validación de esquemas:** Se leyeron los archivos de casos de dengue (104.46 MB) e IPRESS (8.18 MB) aplicando inferencia de esquema y codificación UTF-8. El motor reconoció correctamente los tipos de datos clave (ej. año, ubigeo como *integer* y edad como *double*), lo cual es crítico para las operaciones matemáticas posteriores.

- **Optimización en memoria:** Para evitar latencias de lectura de disco en consultas iterativas, se materializó el DataFrame principal en la memoria RAM mediante el comando `.cache()`, confirmando la carga exacta de 1,029,421 registros. Asimismo, se habilitó una vista temporal para permitir el uso del motor Spark SQL.

```
import os
import sys
import time
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

# Rutas de los archivos limpios
RUTA_CASOS = r"C:\DatasetsLimpios\Casos_Dengue_Nacional.csv"
RUTA_IPRESS = r"C:\DatasetsLimpios\Establecimientos_Nacional.csv"

# Verificar que los archivos existan
for ruta in [RUTA_CASOS, RUTA_IPRESS]:
    if not os.path.isfile(ruta):
        raise FileNotFoundError(f"No se encontró el archivo: {ruta}")
    print(f"Archivo encontrado: {ruta}")

# Crear la sesión de Apache Spark
spark = (
    SparkSession.builder
        .appName("AnálisisBigDataDenguePeru")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "8")
        .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")

print("\n=== ENTORNO DE EJECUCIÓN ===")
print("Python:", sys.version.split()[0])
print("Spark:", spark.version)
print("Modo de ejecución: local[*]")

# Cargar los archivos CSV
casos_df = (
    spark.read
        .option("header", True)
        .option("inferSchema", True)
        .option("encoding", "UTF-8")
        .csv(RUTA_CASOS)
)
```

```
ipress_df = (  
    spark.read  
    .option("header", True)  
    .option("inferSchema", True)  
    .option("encoding", "UTF-8")  
    .csv(RUTA_IPRESS)  
)  
  
print("\nArchivos cargados correctamente.")
```

```
[2] ✓ 16.2s  
... Archivo encontrado: C:\DatasetsLimpios\Casos_Dengue_Nacional.csv  
    Archivo encontrado: C:\DatasetsLimpios\Establecimientos_Nacional.csv  
  
    === ENTORNO DE EJECUCIÓN ===  
    Python: 3.14.8  
    Spark: 4.2.0  
    Modo de ejecución: local[*]  
  
    Archivos cargados correctamente.
```

```
print("===== CASOS DE DENGUE =====")  
  
print("\nColumnas disponibles:")  
print(casos_df.columns)  
  
print("\nEsquema de los datos:")  
casos_df.printSchema()  
  
print("\nPrimeros 5 registros:")  
casos_df.show(5, truncate=False)  
  
print("\n===== ESTABLECIMIENTOS DE SALUD =====")  
  
print("\nColumnas disponibles:")  
print(ipress_df.columns)  
  
print("\nEsquema de los datos:")  
ipress_df.printSchema()  
  
print("\nPrimeros 5 registros:")  
ipress_df.show(5, truncate=False)
```

```
Python
===== CASOS DE DENGUE =====

Columnas disponibles:
['departamento', 'provincia', 'distrito', 'localidad', 'enfermedad', 'ano', 'semana', 'diagnostic', 'diresa', 'ubigeo', 'localcod', 'edad', 'tipo_edad', 'sexo']

Esquema de los datos:
root
|-- departamento: string (nullable = true)
|-- provincia: string (nullable = true)
|-- distrito: string (nullable = true)
|-- localidad: string (nullable = true)
|-- enfermedad: string (nullable = true)
|-- ano: integer (nullable = true)
|-- semana: integer (nullable = true)
|-- diagnostic: string (nullable = true)
|-- diresa: integer (nullable = true)
|-- ubigeo: integer (nullable = true)
|-- localcod: string (nullable = true)
|-- edad: double (nullable = true)
|-- tipo_edad: string (nullable = true)
|-- sexo: string (nullable = true)

Primeros 5 registros:
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|departamento|provincia|distrito|localidad|enfermedad|ano|semana|diagnostic|diresa|ubigeo|localcod|edad|tipo_edad|sexo|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|LORETO      |ALTO AMAZONAS|YURIMAGUAS|000101|DENGUE SIN SIGNOS DE ALARMA|2000|1|A97.0|16|160201|NULL|58.0|A|M|
|LORETO      |ALTO AMAZONAS|YURIMAGUAS|000101|DENGUE SIN SIGNOS DE ALARMA|2000|1|A97.0|16|160201|NULL|44.0|A|F|
```

```
# 7.3. Preparación del conjunto de datos para el análisis

# Mantener los datos en memoria para reducir el costo de
# volver a leer el CSV durante las consultas comparativas.
casos_df.cache()

# Materializar la caché antes de medir las consultas.
total_verificado = casos_df.count()

# Registrar el DataFrame como una vista SQL temporal.
casos_df.createOrReplaceTempView("casos_dengue")

print("Conjunto de datos preparado correctamente.")
print(f"Total de registros verificados: {total_verificado},}")
print("Vista temporal creada: casos_dengue")

[4] ✓ 2.5s

... Conjunto de datos preparado correctamente.
Total de registros verificados: 1,029,421
Vista temporal creada: casos_dengue
```

7.2. Evaluación de rendimiento y comparativa tecnológica

Se sometió a prueba la capacidad de procesamiento del clúster local frente a un motor relacional tradicional (SQL Server Management Studio 22), utilizando consultas de recuento total y agrupaciones iterativas:

- **Velocidad de recuento en memoria:** El escaneo y conteo del millón de registros de dengue tomó apenas **0.160 segundos**, y los 20,793 registros de IPRESS, 0.126 segundos.
- **Spark DataFrames vs. Spark SQL:** Al realizar una agregación anual, el planificador de Spark (*Catalyst Optimizer*) demostró mayor eficiencia al procesar la sintaxis SQL (0.2605 s en promedio) frente a la sintaxis de DataFrames (0.5178 s en promedio).

- **Comparativa con SQL Server:** La misma consulta de agregación anual se ejecutó tres veces en SSMS, promediando un *elapsed time* de **0.2626 segundos**. Spark SQL (0.2605 s) logró igualar e incluso superar marginalmente este tiempo, demostrando que al procesar en RAM, una arquitectura Big Data es altamente competitiva incluso en volúmenes de datos que tradicionalmente dominan las bases de datos relacionales.

```
import time
from pyspark.sql import functions as F

REPETICIONES = 3
# A. Consulta mediante DataFrames
def consulta_dataframe():
    return (
        casos_df
        .groupBy("ano")
        .agg(F.count("*").alias("total_casos"))
        .orderBy("ano")
    )

# B. Consulta mediante Spark SQL
def consulta_spark_sql():
    return spark.sql("""
        SELECT
            ano,
            COUNT(*) AS total_casos
        FROM casos_dengue
        GROUP BY ano
        ORDER BY ano
    """)

# Función para medir una consulta que devuelve resultados
def medir_consulta(funcion, repeticiones=REPETICIONES):
    tiempos = []
    resultado = None

    for _ in range(repeticiones):
        inicio = time.perf_counter()
        resultado = funcion().collect()
        fin = time.perf_counter()
        tiempos.append(fin - inicio)

    return tiempos, resultado
```

```
# Ejecutar y medir ambos métodos
tiempos_df, resultado_df = medir_consulta(consulta_dataframe)
tiempos_sql, resultado_sql = medir_consulta(consulta_spark_sql)

# Mostrar resultados anuales
print("===== CASOS POR AÑO =====")
print("Resultados de DataFrames:")
for fila in resultado_df:
    print(f"Año: {fila['ano']} | Casos: {fila['total_casos']:,}")

# Verificar que ambas consultas produzcan los mismos resultados
equivalentes = resultado_df == resultado_sql

print("\n===== VALIDACIÓN =====")
print("¿Ambas consultas producen los mismos resultados?:",
equivalentes)
print("Años agrupados:", len(resultado_df))

# Mostrar tiempos individuales y promedio
print("\n===== TIEMPOS DE EJECUCIÓN =====")

print("\nDataFrames:")
for i, tiempo in enumerate(tiempos_df, start=1):
    print(f"Ejecución {i}: {tiempo:.4f} segundos")
print(f"Promedio DataFrames: {sum(tiempos_df) / len(tiempos_df):.4f} segundos")

print("\nSpark SQL:")
for i, tiempo in enumerate(tiempos_sql, start=1):
    print(f"Ejecución {i}: {tiempo:.4f} segundos")
print(f"Promedio Spark SQL: {sum(tiempos_sql) / len(tiempos_sql):.4f} segundos")
```

```
... Año: 2013 | Casos: 13,091
    Año: 2014 | Casos: 17,234
    Año: 2015 | Casos: 35,816
    Año: 2016 | Casos: 25,160
    Año: 2017 | Casos: 68,290
    Año: 2018 | Casos: 4,698
    Año: 2019 | Casos: 15,287
    Año: 2020 | Casos: 47,932
    Año: 2021 | Casos: 44,791
    Año: 2022 | Casos: 63,217
    Año: 2023 | Casos: 256,641
    Año: 2024 | Casos: 271,531

===== VALIDACIÓN =====
¿Ambas consultas producen los mismos resultados?: True
Años agrupados: 25

===== TIEMPOS DE EJECUCIÓN =====

DataFrames:
Ejecución 1: 0.9333 segundos
Ejecución 2: 0.3464 segundos
Ejecución 3: 0.2737 segundos
Promedio DataFrames: 0.5178 segundos

Spark SQL:
Ejecución 1: 0.3553 segundos
Ejecución 2: 0.2239 segundos
Ejecución 3: 0.2022 segundos
Promedio Spark SQL: 0.2605 segundos
```

```
# 7.5. Recuento de registros de los conjuntos de datos

inicio = time.perf_counter()
total_casos = casos_df.count()
tiempo_conteo_casos = time.perf_counter() - inicio

inicio = time.perf_counter()
total_ipress = ipress_df.count()
tiempo_conteo_ipress = time.perf_counter() - inicio

print("===== RESULTADOS DEL CONTEO =====")
print(f"Registros de dengue: {total_casos:,}")
print(f"Tiempo de conteo de dengue: {tiempo_conteo_casos:.3f} segundos")

print(f"\nEstablecimientos de salud: {total_ipress:,}")
print(f"Tiempo de conteo de establecimientos: {tiempo_conteo_ipress:.3f} segundos")
```

[6] ✓ 0.2s

```
... ===== RESULTADOS DEL CONTEO =====
Registros de dengue: 1,029,421
Tiempo de conteo de dengue: 0.160 segundos

Establecimientos de salud: 20,793
Tiempo de conteo de establecimientos: 0.126 segundos
```

```
224
225 ---Paso 7---
226 USE DengueCapacidadSanitaria;
227 GO
228
229 SET NOCOUNT ON;
230 SET STATISTICS TIME ON;
231
232 SELECT
233     ano,
234     COUNT_BIG(*) AS total_casos
235 FROM dbo.Caso_Dengue
236 GROUP BY ano
237 ORDER BY ano;
238
239 SET STATISTICS TIME OFF;
240 GO
```

100 % ✓ No se encontraron problemas.

Resultados Mensajes

	ano	total_casos
9	2008	12824
10	2009	13407
11	2010	16842
12	2011	28084
13	2012	28505
14	2013	13091

Validacione...G\User (81))* ✕

```
224
225 ---Paso 7---
226 USE DengueCapacidadSanitaria;
227 GO
228
229 SET NOCOUNT ON;
230 SET STATISTICS TIME ON;
231
232 SELECT
233     ano,
234     COUNT_BIG(*) AS total_casos
235 FROM dbo.Caso_Dengue
236 GROUP BY ano
237 ORDER BY ano;
238
239 SET STATISTICS TIME OFF;
240 GO
```

100 % ✓ No se encontraron problemas.

Resultados Mensajes

SQL Server Execution Times:
CPU time = 234 ms, elapsed time = 283 ms.

Hora de finalización: 2026-10-09T08:24:55.7305996-05:00

```
239 SET STATISTICS TIME OFF;
240 GO
```

121 % ✓ No se encontraron problemas.

Resultados Mensajes

SQL Server Execution Times:
CPU time = 235 ms, elapsed time = 263 ms.

Hora de finalización: 2026-10-09T08:43:38.8607529-05:00

```
239 SET STATISTICS TIME OFF;
240 GO
```

121 % ✓ No se encontraron problemas.

Resultados Mensajes

SQL Server Execution Times:
CPU time = 187 ms, elapsed time = 242 ms.

Hora de finalización: 2026-10-09T08:45:10.6933023-05:00

7.3. Análisis descriptivo y hallazgos epidemiológicos

El procesamiento masivo permitió consolidar el perfil demográfico, temporal y geográfico de los casos de dengue registrados entre 2000 y 2024. Los indicadores porcentuales calculados revelaron lo siguiente:

- **Evolución crítica temporal:** Los años 2023 (256,641 casos) y 2024 (271,531 casos) concentran por sí solos **más del 51% de los registros históricos**. Este crecimiento masivo reciente justifica la transición hacia tecnologías escalables de procesamiento.
- **Carga sobre la Capacidad Sanitaria:** El 88.91% de los registros corresponde a dengue sin signos de alarma, lo que concentra la presión asistencial primaria en los centros de salud (IPRESS). No obstante, el 11.09% acumulado de casos con signos de alarma o graves representa a más de 114,000 pacientes con potencial necesidad de internamiento.
- **Geografía:** Piura (237,961), Lima (122,854) y Loreto (107,074) lideran la estadística histórica. Destaca la alta concentración en la capital, evidenciando el avance urbano de la enfermedad.
- **Perfil Demográfico:** La infección tiene una mayor prevalencia en mujeres (53.79% frente a 46.21% en hombres), con una edad promedio nacional de 30.6 años (población económicamente activa).

```
# 7.7. Análisis de casos por departamento

from pyspark.sql import functions as F

casos_por_departamento = (
    casos_df
    .groupBy("departamento")
    .agg(
        F.count("*").alias("total_casos"),
        F.countDistinct("ubigeo").alias("ubigeos_registrados")
    )
    .orderBy(F.desc("total_casos"))
)

print("===== TOP 15 DEPARTAMENTOS =====")
casos_por_departamento.show(15, truncate=False)

print("\n===== RESUMEN DE LA DISTRIBUCIÓN =====")
print("Departamentos o valores geográficos distintos:")
print(
    casos_df
    .select("departamento")
    .distinct()
    .count()
)

print("Total de registros:")
print(casos_df.count())
```

[7] ✓ 1.9s

```
print(casos_df.count())
```

[7] ✓ 1.9s

```
... ===== TOP 15 DEPARTAMENTOS =====
+-----+-----+-----+
|departamento|total_casos|ubigeos_registrados|
+-----+-----+-----+
|PIURA      |237961     |62                  |
|LIMA        |122854     |81                  |
|LORETO      |107074     |53                  |
|LA LIBERTAD |100810     |44                  |
|ICA         |68620      |39                  |
|LAMBAYEQUE  |56383      |37                  |
|UCAYALI     |52621      |19                  |
|TUMBES      |45471      |13                  |
|SAN MARTIN  |44518      |78                  |
|ANCASH      |37661      |17                  |
|MADRE DE DIOS|32920      |11                  |
|CAJAMARCA   |29711      |53                  |
|JUNIN       |26110      |19                  |
|AMAZONAS    |16805      |25                  |
|CUSCO       |15210      |21                  |
+-----+-----+-----+
only showing top 15 rows

===== RESUMEN DE LA DISTRIBUCIÓN =====
Departamentos o valores geográficos distintos:
23
Total de registros:
1029421
```

```
# 7.8. Análisis de casos según diagnóstico

diagnosticos = (
    casos_df
    .groupBy("diagnostic", "enfermedad")
    .agg(F.count("*").alias("total_registros"))
    .orderBy(F.desc("total_registros"))
)

print("===== DISTRIBUCIÓN POR DIAGNÓSTICO =====")
diagnosticos.show(20, truncate=False)

print("\n===== CANTIDAD DE CATEGORÍAS =====")
print("Códigos de diagnóstico distintos:")
print(casos_df.select("diagnostic").distinct().count())

print("Descripciones de enfermedad distintas:")
print(casos_df.select("enfermedad").distinct().count())
```

✓ 1.0s

```
===== DISTRIBUCIÓN POR DIAGNÓSTICO =====
+-----+-----+-----+
|diagnostic|enfermedad|total_registros|
+-----+-----+-----+
|A97.0     |DENGUE SIN SIGNOS DE ALARMA|915243|
|A97.1     |DENGUE CON SIGNOS DE ALARMA|110166|
|A97.2     |DENGUE GRAVE|4012|
+-----+-----+-----+

===== CANTIDAD DE CATEGORÍAS =====
Códigos de diagnóstico distintos:
3
Descripciones de enfermedad distintas:
3
```

```
print("===== REGISTROS POR UNIDAD DE EDAD =====")

casos_df.groupBy("tipo_edad").agg(
    F.count("*").alias("total_registros")
).orderBy(
    F.desc("total_registros")
).show(truncate=False)
```

[9] ✓ 1.0s

```
...
===== ESTADÍSTICAS DE EDAD =====
+-----+-----+-----+-----+-----+
|edades_no_nulas|edad_promedio|desviacion_estandar|edad_minima|edad_maxima|
+-----+-----+-----+-----+-----+
|1029407|30.596995428435985|18.996258511396444|0.0|110.0|
+-----+-----+-----+-----+-----+

===== REGISTROS POR SEXO =====
+-----+-----+
|sexo|total_registros|
+-----+-----+
|F|553677|
|M|475744|
+-----+-----+

===== REGISTROS POR UNIDAD DE EDAD =====
+-----+-----+
|tipo_edad|total_registros|
+-----+-----+
|A|1029421|
+-----+-----+
```

```
)
```

[10] ✓ 1.3s

```
...
===== PARTICIPACIÓN POR DIAGNÓSTICO =====
+-----+-----+-----+-----+
|diagnostic|enfermedad|registros|porcentaje|
+-----+-----+-----+-----+
|A97.0|DENGUE SIN SIGNOS DE ALARMA|915243|88.91|
|A97.1|DENGUE CON SIGNOS DE ALARMA|110166|10.7|
|A97.2|DENGUE GRAVE|4012|0.39|
+-----+-----+-----+-----+

===== AÑOS CON MÁS REGISTROS =====
+-----+-----+
|ano|registros|
+-----+-----+
|2024|271531|
|2023|256641|
|2017|68290|
|2022|63217|
|2020|47932|
+-----+-----+
only showing top 5 rows

===== PARTICIPACIÓN POR SEXO =====
+-----+-----+-----+
|sexo|registros|porcentaje|
+-----+-----+-----+
|F|553677|53.79|
|M|475744|46.21|
+-----+-----+-----+
```

7.4. Inspección física de ejecución y escalabilidad

Finalmente, se evaluó el comportamiento algorítmico subyacente para validar la escalabilidad de la solución:

- **Paralelismo simétrico:** Spark segmentó los 104.46 MB del archivo de casos en exactamente **12 particiones**, alineándose de forma óptima con los 12 núcleos del procesador, logrando un uso total del hardware.
- **Optimización del plan físico:** El plan de ejecución (Physical Plan) evidenció nodos de lectura pura en memoria (InMemoryTableScan) sin acceso a disco duro. Asimismo, la operación de intercambio (Exchange) reflejó el reajuste del particionamiento *hash* a solo 8 salidas, confirmando que la configuración manual evitó la saturación por sobre-distribución (*shuffle overhead*).

```
# 7.12. Inspección del plan de ejecución y particiones

import os

print("===== TAMAÑO DE LOS ARCHIVOS =====")
print(
    "CSV de dengue:",
    f"{os.path.getsize(RUTA_CASOS) / (1024**2):.2f} MB"
)
print(
    "CSV de establecimientos:",
    f"{os.path.getsize(RUTA_IPRESS) / (1024**2):.2f} MB"
)

print("\n===== PARTICIONES DEL DATAFRAME =====")
print("Particiones de casos df:", casos_df.rdd.getNumPartitions())
print("Núcleos lógicos disponibles para Spark:", spark.sparkContext.defaultParallelism)

print("\n===== PLAN DE EJECUCIÓN DE LA AGREGACIÓN =====")

(
    casos_df
    .groupBy("ano")
    .agg(F.count("*").alias("total_casos"))
    .explain("formatted")
)
```

```
===== TAMAÑO DE LOS ARCHIVOS =====
CSV de dengue: 104.46 MB
CSV de establecimientos: 8.18 MB

===== PARTICIONES DEL DATAFRAME =====
Particiones de casos df: 12
Núcleos lógicos disponibles para Spark: 12

===== PLAN DE EJECUCIÓN DE LA AGREGACIÓN =====
== Physical Plan ==
AdaptiveSparkPlan (7)
+- HashAggregate (6)
   +- Exchange (5)
      +- HashAggregate (4)
         +- InMemoryTableScan (1)
            +- InMemoryRelation (2)
               +- Scan csv (3)

(1) InMemoryTableScan
Output [1]: [ano#22]
Arguments: [ano#22]

(2) InMemoryRelation
Arguments: [departamento#17, provincia#18, distrito#19, localidad#20, enfermedad#21, ano#22, semana#23, diagnostic#24, diresa#25, ubigeo#26, localcod#27, edad#28, tipo_edad#29, sexo#30]

(3) Scan csv
Output [14]: [departamento#17, provincia#18, distrito#19, localidad#20, enfermedad#21, ano#22, semana#23, diagnostic#24, diresa#25, ubigeo#26, localcod#27, edad#28, tipo_edad#29, sexo#30]
Batched: false
```

```
[11] ✓ 0.0s Python
...
(3) Scan csv
Output [14]: [departamento#17, provincia#18, distrito#19, localidad#20, enfermedad#21, ano#22, semana#23, diagnostic#24, diresa#25, ubigeo#26, localcod#27, edad#28, tipo_edad#29, se
Batched: false
Location: InMemoryFileIndex [file:/c:/Datasets/limpios/Casos_Dengue_Nacional.csv]
ReadSchema: struct<departamento:string,provincia:string, distrito:string, localidad:string, enfermedad:string, ano:int, semana:int, diagnostic:string, diresa:int, ubigeo:int, localcod:string

(4) HashAggregate
Input [1]: [ano#22]
Keys [1]: [ano#22]
Functions [1]: [partial_count(1)]
Aggregate Attributes [1]: [count#8738L]
Results [2]: [ano#22, count#8739L]

(5) Exchange
Input [2]: [ano#22, count#8739L]
Arguments: hashpartitioning(ano#22, 8), ENSURE_REQUIREMENTS, [plan_id=1898]

(6) HashAggregate
Input [2]: [ano#22, count#8739L]
Keys [1]: [ano#22]
Functions [1]: [count(1)]
Aggregate Attributes [1]: [count(1)#8527L]
Results [2]: [ano#22, count(1)#8527L AS total_casos#8512L]

(7) AdaptiveSparkPlan
Output [2]: [ano#22, total_casos#8512L]
Arguments: isFinalPlan=false
```

7.5. Conclusiones del capítulo

- **Consistencia y Procesamiento:** Apache Spark permitió procesar el conjunto de datos de dengue, obtener indicadores descriptivos y ejecutar consultas equivalentes mediante DataFrames y Spark SQL. Los resultados de la agregación anual coincidieron exactamente con los de SQL Server, lo que respalda la consistencia e integridad de los datos en esta prueba.
- **Rendimiento y Escalabilidad:** Los tiempos medidos muestran que Spark SQL y SQL Server tuvieron un rendimiento muy similar en la consulta evaluada. No obstante, las condiciones de ejecución y el uso de la memoria caché limitan la posibilidad de generalizar esta diferencia a otros escenarios. Aunque Spark ofrece mecanismos superiores para particionar y paralelizar, una conclusión definitiva sobre escalabilidad requerirá pruebas adicionales con mayores volúmenes o en un entorno de red distribuido.
- **Valor Epidemiológico:** La experiencia permite valorar la gran utilidad de Spark para la salud pública. La herramienta facilitó la extracción rápida de indicadores críticos, evidenciando que el 51% de los registros históricos se concentran solo en 2023-2024 y que el 88.9% de la presión recae sobre el primer nivel de atención. Esto justifica técnica y operativamente la necesidad de transitar hacia arquitecturas Big Data ante el crecimiento exponencial de la información sanitaria.

CAPÍTULO VIII

INTEGRACIÓN, REFLEXIÓN ÉTICA Y CONCLUSIONES DEL PROYECTO

8.1. Integración analítica: tecnología y salud pública

El proyecto permitió aplicar tecnologías de bases de datos, análisis de datos y Big Data al estudio de los registros históricos de dengue en el Perú entre 2000 y 2024. Se procesaron más de un millón de registros mediante Apache Spark y se analizaron datos de establecimientos de salud (IPRESS), generando información útil para comprender la distribución temporal y geográfica de los registros epidemiológicos.

8.2. Reflexión ética en el manejo de datos

El análisis se centró en variables epidemiológicas y demográficas, evitando exponer información personal identificable en los resultados presentados. Asimismo, se reconoce que los recuentos analizados corresponden a registros notificados y no necesariamente representan tasas de incidencia poblacional. Por ello, los resultados deben interpretarse con responsabilidad y utilizarse como apoyo para el análisis, sin sustituir la evaluación de los especialistas de salud pública.

8.3. Transparencia y reproducibilidad

El código desarrollado, las consultas y los resultados documentados en el notebook permiten revisar el procedimiento aplicado. Para facilitar la reproducibilidad, se recomienda conservar los archivos utilizados, las instrucciones de ejecución y las versiones de las herramientas.

Repositorio en GitHub: [<https://github.com/OsvaldoGuerra/analisis-big-data-dengue-peru>].

8.4. Conclusiones generales

- **Viabilidad tecnológica:** Apache Spark permitió procesar más de un millón de registros y realizar agregaciones mediante DataFrames y Spark SQL. En las pruebas realizadas, Spark SQL y SQL Server obtuvieron tiempos promedio muy similares.

- **Comportamiento temporal:** Los años 2023 y 2024 concentraron aproximadamente el 51.3 % de los registros analizados. Este resultado evidencia un volumen elevado de notificaciones durante esos años, aunque no permite explicar por sí solo sus causas.
- **Distribución de los diagnósticos:** El 88.91 % de los registros correspondió a dengue sin signos de alarma. Este hallazgo describe la composición de los datos y puede servir como punto de partida para estudios posteriores sobre la atención sanitaria.
- **Valor del análisis de datos:** El uso combinado de herramientas de bases de datos, visualización y Big Data permitió organizar y analizar información epidemiológica y de establecimientos de salud, sentando una base para futuras investigaciones que relacionen los casos de dengue con la capacidad de atención sanitaria.